

操作系统课程设计

xv6 实验报告

题目：基于 Xv6_Labs_Operation_System_2026Summer 提交仓库的 RISC-V xv6 实验实现

实验范围：util / syscall / pgtbl / traps / cow / thread / net / lock / fs / mmap

姓名：阮文涛 学号：2350757 班级：03

日期：2026-07-22

目录

目录	2
摘要	5
实验环境与材料目录	5
分支与代码位置总览	5
实验零 环境配置与版本核查	7
实验目的	7
环境对照	7
配置步骤	8
遇到的问题和解决方法	8
环境配置心得	8
实验一 Xv6 and Unix utilities.....	9
Boot xv6 (easy).....	9
sleep (easy)	10
pingpong (easy)	12
primes (moderate / hard).....	13
find (moderate)	14
xargs (moderate).....	16
Lab1 总测试结果	18
实验二 System calls	19
trace	19
sysinfo.....	20
Lab2 总测试结果	22
实验三 Page Tables.....	23
Speed up system calls / ugetpid	23
Print a page table	24
Detect which pages have been accessed.....	25
Lab3 总测试结果	27
实验四 Traps	28
RISC-V backtrace.....	28
Alarm.....	29
Lab4 总测试结果	30
实验五 Copy-on-write Fork	32
COW fork	32
copyout and usertests.....	33

Lab5 总测试结果	34
实验六 Multithreading	36
Uthread	36
Using threads	37
Barrier	38
Lab6 总测试结果	40
实验七 Network Driver	41
E1000 transmit	41
E1000 receive	42
Lab7 总测试结果	43
实验八 Locks	45
Memory allocator	45
Buffer cache	46
Lab8 总测试结果	48
实验九 File System	49
Large files	49
Symbolic links	50
Lab9 总测试结果	52
实验十 Mmap	53
mmap	53
munmap and page fault	54
Lab10 总测试结果	56
综合验证与复现说明	57
共性问题与调试分析	57
生命周期对称	57
权限位与状态位	57
并发语义	57
测试复核	58
项目成果说明	58
总结	58
附录 A: patch 文件清单	59
附录 B: 主要源码摘录	60
Lab1 - sleep.c	60
Lab1 - pingpong.c	60
Lab1 - primes.c	61
Lab1 - find.c	62

Lab1 - xargs.c	63
Lab2 - trace 与 sysinfo	64
Lab3 - pgaccess	65
Lab4 - backtrace	65
Lab4 - sigalarm 与 sigreturn.....	66
Lab5 - COW 核心路径	66
Lab6 - 用户级线程.....	68
Lab6 - barrier.....	71
Lab7 - e1000 网卡驱动	71
Lab8 - per-CPU allocator	72
Lab8 - buffer cache.....	73
Lab9 - large files.....	75
Lab9 - symbolic links	76
Lab10 - mmap/munmap.....	76
附录 C: 参考资料	错误!未定义书签。

摘要

本课程设计依据《xv6 及 Labs 课程项目（2026）》要求，以 MIT 6.S081 官方 xv6 Labs 实验路线为基础，在 `XV6_Labs_Operation_System_2026Summer` 提交仓库中完成 `util`、`syscall`、`pgtbl`、`traps`、`cow`、`thread`、`net`、`lock`、`fs`、`mmap` 共十个实验。报告重点记录实验目的、实验内容、原理设计、关键代码、实现步骤、测试方案、问题分析和结果记录。

在实验过程中，代码实现、分支管理、补丁文件、测试日志和运行截图均按实验顺序整理。每个实验都先分析官方要求和 xv6 相关源码，再完成代码修改，最后通过 `make grade` 或对应子测试进行验证。

实验环境与材料目录

项目	内容
实现仓库	GitHub: https://github.com/jyybacpg/XV6_Labs_Operation_System_2026Summer.git
材料目录	实验报告、测试截图、patch 文件和主要源码摘录
代码分支	<code>solution-util</code> , <code>solution-syscall</code> , <code>solution-pgtbl</code> , <code>solution-traps</code> , <code>solution-cow</code> , <code>solution-thread</code> , <code>solution-net</code> , <code>solution-lock</code> , <code>solution-fs</code> , <code>solution-mmap</code>
可复现材料	GitHub 2026 仓库 <code>solution-*</code> 分支、测试命令、测试截图、patch/源码摘录
推荐运行环境	Ubuntu / WSL Ubuntu / Docker，安装 <code>git</code> 、 <code>build-essential</code> 、 <code>gdb-multiarch</code> 、 <code>qemu-system-misc</code> 、 <code>ipxe-qemu</code> 、 <code>gcc-riscv64-unknown-elf</code>

分支与代码位置总览

Lab	分支	Commit	相关代码位置	实现摘要
util	<code>solution-util</code>	0220fd0	<code>user/sleep.c</code> ; <code>user/pingpong.c</code> ; <code>user/primes.c</code> ; <code>user/find.c</code> ; <code>user/xargs.c</code> ; <code>Makefile</code>	完成用户态工具程序，覆盖进程、管道、递归文件遍历和参数批处理。
syscall	<code>solution-syscall</code>	13a2f9e	<code>kernel/sysproc.c</code> ; <code>kernel/syscall.c</code> ; <code>kernel/proc.c</code> ; <code>kernel/kalloc.c</code> ; <code>user/trace.c</code>	新增 <code>trace</code> 和 <code>sysinfo</code> 系统调用，补齐用户态 <code>stub</code> 、内核分派、进程继承和信息拷贝。
pgtbl	<code>solution-pgtbl</code>	d27dab0	<code>kernel/proc.c</code> ; <code>kernel/vm.c</code> ; <code>kernel/sysproc.c</code> ; <code>kernel/riscv.h</code>	实现 <code>USYSCALL/ugetpid</code> 、 <code>vmprint</code> 和 <code>pgaccess</code> ，

				练习 Sv39 页表遍历与 PTE 标志维护。
traps	solution-traps	7e3990f	kernel/trap.c; kernel/proc.h; kernel/sysproc.c; kernel/riscv.h	实现 backtrace、sigalarm、sigreturn，并保存/恢复用户陷入现场。
cow	solution-cow	e09d1c9	kernel/vm.c; kernel/kalloc.c; kernel/trap.c; kernel/riscv.h	实现 copy-on-write fork、物理页引用计数、写缺页复制和 copyout COW 处理。
thread	solution-thread	557ce66	user/uthread.c; user/uthread_switch.S; notxv6/ph.c; notxv6/barrier.c	实现用户级线程切换、哈希表桶锁优化和 pthread barrier。
net	solution-net	33ee4c4	kernel/e1000.c; kernel/net.c; kernel/pci.c	实现 E1000 描述符环的发送与接收，处理 mbuf 生命周期和锁边界。
lock	solution-lock	5ebad33	kernel/kalloc.c; kernel/bio.c	将物理页分配器拆成 per-CPU freelist，将 bcache 改为哈希桶锁以降低竞争。
fs	solution-fs	7c6dae2	kernel/fs.h; kernel/file.h; kernel/fs.c; kernel/sysfile.c; user/usys.pl	实现大文件双重间接块、symlink 系统调用、O_NOFOLLOW 和 open 链接解析。
mmap	solution-mmap	ea8b72c	kernel/proc.h; kernel/sysfile.c; kernel/trap.c; kernel/proc.c; kernel/exec.c	实现 mmap/munmap、VMA 表、按需文件页装入、MAP_SHARED 写回和生命周期清理。

实验零 环境配置与版本核查

本章说明

- 本章说明 `XV6_Labs_Operation_System_2026Summer` 提交仓库的编译、运行、调试和评分环境。
- 本项目依赖 `Linux`、`QEMU` 和 `RISC-V` 工具链，并采用 `Docker/Ubuntu` 完成真实动态测试。
- 实验范围严格按课程要求的十个 `xv6 Lab` 组织，不额外并入其他年份的独立实验。

实验目的

- 建立可复现的 `XV6_Labs_Operation_System_2026Summer` 提交仓库编译、运行、调试和评分环境。
- 确认 `QEMU`、`RISC-V` 交叉编译器、`GNU Make`、`Git` 和网络实验依赖均可用。
- 记录本次实验的测试日志、截图和相关材料，使实验结果能够被复核。
- 区分 `WSL/Ubuntu`、`Docker/Ubuntu` 和普通 `Linux` 虚拟机路线的共同点与差异。

环境对照

项目	本项目实际配置	说明
实验主机	Windows + Ubuntu 工具链环境	用于编辑源码、运行 Ubuntu 环境并保存测试日志。
Linux 环境	Ubuntu 容器环境	执行 <code>make</code> 、 <code>QEMU</code> 、 <code>RISC-V GCC</code> 、 <code>grade</code> 脚本，保证测试在类 Linux 环境中完成。
核心工具链	<code>qemu-system-riscv64</code> 、 <code>riscv64-unknown-elf-gcc</code> 、 <code>GNU Make</code>	满足 <code>XV6_Labs_Operation_System_2026Summer</code> 提交仓库对 <code>RISC-V</code> 模拟器、交叉编译器和构建工具的要求。
补充依赖	<code>gdb-multiarch</code> 、 <code>ipxe-qemu</code> 、 <code>git</code> 、 <code>build-essential</code>	支持调试、网络实验 <code>E1000 ROM</code> 、版本管理和基本编译。
测试产物	<code>real_test_logs</code> 、 <code>browser_screenshots</code> 、 <code>xv6.out</code> 、 <code>packets.pcap</code>	保留完整 <code>make grade</code> 证据链，网络实验额外保存包捕获相关产物。

本实验采用 `Ubuntu` 工具链完成；同样可迁移到 `WSL Ubuntu`、`Linux` 虚拟机或 `Docker Ubuntu` 容器。三种环境的本质相同：都在 `Linux` 用户空间中使用 `QEMU` 模拟 `RISC-V xv6`，并用交叉编译器生成内核和用户程序。

配置步骤

- 1. 安装或准备 Linux 环境：可使用 WSL Ubuntu、Linux 虚拟机，或 Docker Desktop 中的 Ubuntu 容器。
- 2. 安装基础依赖：git、build-essential、GNU Make、QEMU RISC-V、GDB 和 RISC-V GCC。
- 3. 克隆或恢复 XV6_Labs_Operation_System_2026Summer 提交仓库，分别切换到 solution-util、solution-syscall、solution-pgtbl、solution-traps、solution-cow、solution-thread、solution-net、solution-lock、solution-fs、solution-mmap 分支。
- 4. 先运行 make qemu 验证系统能启动，再逐实验运行 make grade 验证实现正确性。

```
sudo apt-get update
sudo apt-get install -y git build-essential gdb-multiarch qemu-system-misc ipxe-
qemu gcc-riscv64-unknown-elf
qemu-system-riscv64 --version
riscv64-unknown-elf-gcc --version
make --version
```

遇到的问题和解决方法

问题	原因分析	解决方法
新版 GCC 把 user/sh.c 的递归警告视作错误	部分 Ubuntu/GCC 13 环境会出现 -Werror=infinite-recursion。	在 grading 运行中加入 -Wno-infinite-recursion 作为工具链兼容参数，不改变 xv6 内核/用户程序语义。
Docker 网络实验中 server.py 仅绑定 localhost	容器网络与 QEMU user networking 的可达性和普通 WSL/物理机不同。	容器内测试时将 server.py 绑定到 0.0.0.0，使 QEMU 网络测试可访问宿主服务。
QEMU 找不到 efi-e1000.rom	Ubuntu 包把 E1000 ROM 放在 ipxe-qemu 路径下，QEMU 默认查找路径不一定一致。	安装 ipxe-qemu，并建立 /usr/share/qemu/efi-e1000.rom 到实际 ROM 文件的软链接。

环境配置心得

环境配置心得：这次环境配置让我认识到，操作系统实验的第一步不是写代码，而是建立稳定、可复现的运行环境。xv6 同时依赖 RISC-V 交叉编译器、QEMU、GNU Make、Git 和评分脚本，任意一个工具版本或路径配置不正确都会影响后续测试。通过整理安装命令、版本核查和仓库分支，我学会了把实验环境当作实验结果的一部分来记录。这个过程提升了我排查工具链问题、区分宿主系统和 xv6 系统命令的能力，也让我在后续每个 Lab 中能更快定位问题来源。

实验一 Xv6 and Unix utilities

本实验从用户态程序入手，完成 sleep、pingpong、primes、find、xargs 五个小程序，并确认它们被 Makefile 打包进 xv6 文件系统镜像。

【本实验任务】

- **Boot xv6 (easy):** 确认 xv6 能在 QEMU 中正常启动和退出，熟悉 xv6 shell 的基本操作，为后续用户程序测试提供入口。
- **sleep (easy):** 实现 xv6 版 sleep 命令，使进程按照用户输入的 tick 数休眠，练习最基本的参数解析和系统调用封装使用。
- **pingpong (easy):** 使用 pipe 和 fork 完成父子进程之间的双向 1 字节通信，理解管道文件描述符和进程同步关系。
- **primes (moderate / hard):** 用管道和多进程递归实现素数筛，体会 Unix 管道把多个小程序串联成计算流水线的思想。
- **find (moderate):** 实现递归目录查找程序，练习 open、read、fstat、dirent 和 stat 在 xv6 文件系统中的配合使用。
- **xargs (moderate):** 实现简化版 xargs，把标准输入按行读取后追加到命令参数末尾，并通过 fork/exec 执行目标命令。

Boot xv6 (easy)

【实验目的】

Boot xv6 (easy) / 启动 xv6。确认 xv6 能够在 QEMU 中启动、进入 shell 并正常退出；熟悉 xv6 的基本交互界面，为后续用户程序的手动测试和评分脚本运行建立统一环境。

【实验步骤】

1. 进入 XV6_Labs_Operation_System_2026Summer 提交仓库，切换到 solution-util 分支。
2. 执行 make clean 清理旧目标文件，再执行 make qemu 启动 xv6。
3. 看到 init: starting sh 和 \$ 提示符后，运行 ls 检查基础命令是否可用。
4. 用 Ctrl-a x 退出 QEMU，确认启动和退出流程正常。

【实验代码】执行命令

```
git checkout solution-util
make clean
make qemu

# xv6 shell
ls

# 退出 QEMU: Ctrl-a 后按 x
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-util origin/solution-util
make clean
make qemu
# xv6 shell 中执行
ls
# 退出 QEMU: Ctrl-a 后按 x
```

```

riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
== Test sleep, no arguments == sleep, no arguments: OK (2.6s)
== Test sleep, returns == sleep, returns: OK (0.6s)
== Test sleep, makes syscall == sleep, makes syscall: OK (1.0s)
== Test pingpong == pingpong: OK (1.0s)
== Test primes == primes: OK (1.0s)
== Test find, in current directory == find, in current directory: OK (1.0s)
== Test find, recursive == find, recursive: OK (1.1s)
== Test xargs == xargs: OK (1.1s)
== Test time ==
time: OK
Score: 100/100
sunday@localhost: ~/xv6-labs-2021-solution-run$

```

图 1-1 Boot xv6 启动成功、出现 \$ 提示符并执行 ls 的终端截图

【测试方法与预期输出】

- 预期输出：QEMU 启动后出现 `init: starting sh` 和 `$` 提示符；执行 `ls` 能看到 `README`、`xargstest.sh` 以及已打包进文件系统的用户程序。
- 评分复核：Boot 部分不单独计分；能够进入 xv6 shell 并执行 `ls`，说明后续用户程序可以被打包和运行。

【实验中遇到的问题和解决方法】

- 问题一：第一次进入 QEMU 后不熟悉 xv6 shell 和退出方式，容易把 `Ctrl-a x` 写成普通终端命令。解决时先按课程说明确认启动日志、`$` 提示符和 `ls` 输出，再单独记录退出组合键，保证后续每次测试都能正常收尾。
- 问题二：`make clean` 后重新启动耗时较长，刚开始会误以为卡住。后来通过观察 `init: starting sh` 和文件列表确认系统已经进入用户态，这一步也帮助我分清宿主 Ubuntu 命令和 xv6 shell 命令的执行位置。

【实验心得】

这次实验虽然没有正式修改代码，但让我先建立了完整的 xv6 运行意识。通过反复执行 `make qemu`、观察 `init` 启动信息、进入 shell 并执行 `ls`，我学会了区分宿主 Ubuntu 命令和 xv6 内部命令，也知道了用户程序必须被打包进文件系统后才能运行。这个过程提升了我配置实验环境、判断启动是否成功以及保存复现证据的能力，为后面每个实验的编译、运行和截图打下了基础。

sleep (easy)

【实验目的】

sleep (easy) / Implement the UNIX program sleep for xv6 / 实现 xv6 版 UNIX sleep 程序。实现 `sleep ticks` 命令，使用户输入的 `tick` 参数能够通过系统调用传入内核并使进程休眠；同时练习 `argc/argv` 参数检查、`atoi` 转换和错误用法提示。

【实验步骤】

1. 在 `user` 目录下新增 `sleep.c`，并包含 `kernel/types.h` 与 `user/user.h`。
2. 检查 `argc` 是否等于 2；参数错误时输出 `usage` 并 `exit(1)`。
3. 使用 `atoi(argv[1])` 转换 `tick` 数，调用 `sleep(ticks)`。
4. 在 `Makefile` 的 `UPROGS` 中加入 `$U/_sleep`，使其进入文件系统镜像。

【实验代码】user/sleep.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep ticks\n");
        exit(1);
    }

    sleep(atoi(argv[1]));
    exit(0);
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-util origin/solution-util
make clean
make qemu
# xv6 shell 中执行
sleep 10
echo sleep_done
sleep
```

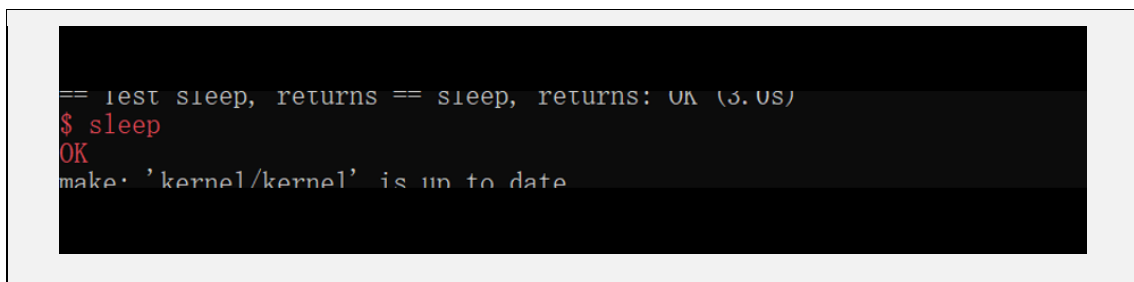


图 1-2 sleep 10 运行结果和 sleep 子测试通过截图

【测试方法与预期输出】

- 预期输出：sleep 10 短暂停顿后返回 shell，echo 输出 sleep_done；无参数运行 sleep 时输出 usage: sleep ticks。
- 评分复核：./grade-lab-util 中应出现 sleep, no arguments、sleep, returns、sleep, makes syscall 均 OK。

【实验中遇到的问题和解决方法】

- 问题一：最初只关注 sleep(atoi(argv[1])), 忽略了参数数量检查，无参数运行时直接访问不存在的 argv[1]。解决方法是在入口处判断 argc，不满足要求就向 2 号文件描述符输出 usage 并 exit(1)。
- 问题二：写完 sleep.c 后在 xv6 中执行提示找不到命令。检查 Makefile 后发现还需要把 _sleep 加入 UPROGS，使程序被打包进 fs.img，重新 make clean 后再测试即可。

【实验心得】

这次实验让我从最简单的用户程序开始理解 xv6 的系统调用使用方式。通过实现 sleep，我学会了用 argc 和 argv 解析命令行参数，知道错误提示应写到 2 号文件描述符，也理解了 sleep(ticks) 实际是通过用户态封装进入内核完成等待。调试过程中我还学会了检查 Makefile 中的 UPROGS，提升了从代码文件、构建脚本到 xv6 shell 命令整体排查问题的能力。

pingpong (easy)

【实验目的】

pingpong (easy) / Use UNIX system calls to ping-pong a byte between two processes / 使用 UNIX 系统调用在两个进程之间传递 1 字节。使用 pipe、fork、read、write 和 wait 完成父子进程双向通信，理解文件描述符读写端、父子进程返回值差异和进程同步顺序。

【实验步骤】

1. 创建两条管道，分别负责 parent -> child 和 child -> parent。
2. fork 后，子进程读取父进程发送的字节，输出 received ping，再写回一个字节。
3. 父进程先写入 ping 字节，再读取子进程回复，输出 received pong。
4. 关闭不使用的管道端口，父进程 wait 等待子进程结束。

【实验代码】 user/pingpong.c

```
int p2c[2], c2p[2];
char buf = 'x';

pipe(p2c);
pipe(c2p);

if(fork() == 0){
    close(p2c[1]);
    close(c2p[0]);
    read(p2c[0], &buf, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &buf, 1);
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
}

close(p2c[0]);
close(c2p[1]);
write(p2c[1], &buf, 1);
read(c2p[0], &buf, 1);
printf("%d: received pong\n", getpid());
close(p2c[1]);
close(c2p[0]);
wait(0);
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-util origin/solution-util
make clean
make qemu
# xv6 shell 中执行
pingpong
```

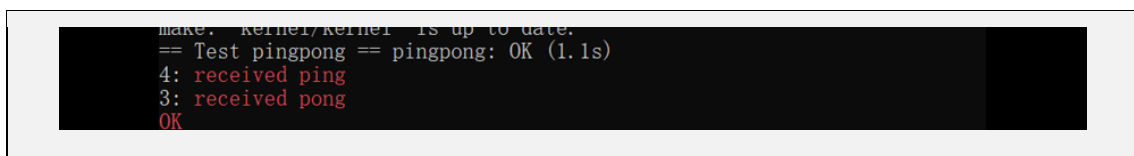


图 1-3 pingpong 输出 ping/pong 的终端截图

【测试方法与预期输出】

- 预期输出：先出现子进程 `pid: received ping`，再出现父进程 `pid: received pong`；两个 `pid` 可以不同，输出顺序应符合父子通信逻辑。
- 评分复核：./grade-lab-util 中 `pingpong` 子测试应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：一开始想用一条 `pipe` 同时完成父子进程双向通信，读写端很容易混在一起。后来改成 `p2c` 和 `c2p` 两条管道，分别表示 `parent-to-child` 和 `child-to-parent`，通信方向清楚后输出顺序也稳定。
- 问题二：如果父进程不 `wait`，终端中有时只能看到一半输出。解决时让父进程在收到 `pong` 后 `wait(0)`，确保子进程打印和退出都完成，测试脚本也能稳定捕获 `ping/pong` 两行。

【实验心得】

这次实验让我更直观地理解了 `fork` 和 `pipe` 在进程通信中的配合方式。通过建立 `p2c` 和 `c2p` 两条管道，我学会了区分管道读端、写端以及父子进程各自应关闭的文件描述符。实现 `ping` 和 `pong` 的过程中，我也认识到 `wait` 对保证输出完整性很重要。完成后，我对 Unix 中小程序如何通过进程和文件描述符协作有了更清楚的认识，进程同步和调试输出顺序的能力也得到了提升。

primes (moderate / hard)

【实验目的】

`primes (moderate/hard)` / Use pipes to implement a prime sieve / 使用管道实现素数筛。用多进程和管道递归实现素数筛，体会 UNIX pipeline 的思想：每一层进程只负责筛掉一个素数的倍数，并把剩余数字交给下一层。

【实验步骤】

1. 父进程向第一条管道写入 2 到 35。
2. 每一层进程读取第一个数作为当前素数并打印。
3. 创建下一条管道和子进程，把不能被当前素数整除的数写给下一层。
4. 所有写端关闭后，下一层 `read` 返回 0，递归自然结束。

【实验代码】user/primes.c

```
static void
relay(int leftfd)
{
    int prime;
    if(read(leftfd, &prime, sizeof(prime)) != sizeof(prime)){
        close(leftfd);
        exit(0);
    }
    printf("prime %d\n", prime);

    int pipefd[2];
    pipe(pipefd);

    if(fork() == 0){
        close(pipefd[1]);
        close(leftfd);
        relay(pipefd[0]);
    }

    close(pipefd[0]);
    int n;
```

```

while(read(leftfd, &n, sizeof(n)) == sizeof(n)){
    if(n % prime != 0)
        write(pipefd[1], &n, sizeof(n));
}
close(leftfd);
close(pipefd[1]);
wait(0);
exit(0);
}

```

// main 中写入 2..35, 第一层 relay 开始递归筛选。

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-util origin/solution-util
make clean
make qemu
# xv6 shell 中执行
primes

```

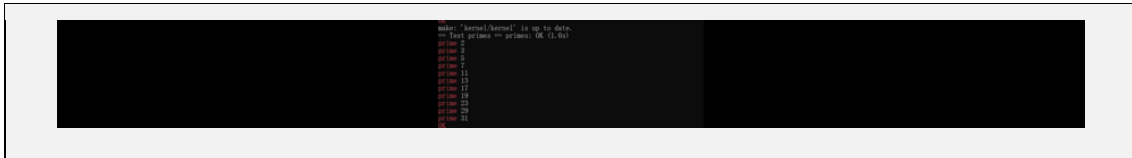


图 1-4 primes 输出 2 到 31 的素数列表截图

【测试方法与预期输出】

- 预期输出：依次输出 prime 2、3、5、7、11、13、17、19、23、29、31，不应输出合数。
- 评分复核：./grade-lab-util 中 primes 子测试应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：调试时最容易漏关管道写端，导致下一层 read 一直等 EOF，程序看起来像卡死。解决时给每一层进程明确关闭不用的读端和写端，让数据流结束后能正确退出递归。
- 问题二：递归筛看起来像普通素数筛，但它依赖多个进程级联过滤。实现时我先固定输入 2 到 35，再观察每层取出的第一个数就是 prime，确认只有不能被当前 prime 整除的数才写入右侧管道。

【实验心得】

这次实验让我理解了 xv6 中管道可以被组织成递归的数据处理链。primes 不是普通数组筛法，而是每一层进程读取一个 prime，再把不能被它整除的数字传给下一层进程。实现时我学会了通过关闭写端让 read 得到 EOF，避免程序一直阻塞。这个实验提升了我分析多进程递归、判断管道生命周期以及用 Unix 思想拆分任务的能力，也让我感受到 xv6 用户程序和真实系统机制之间的联系。

find (moderate)

【实验目的】

find (moderate) / Write a simple version of the UNIX find program / 实现简化版 UNIX find。实现递归目录查找，练习 open、read、fststat、stat、dirent 和路径拼接，理解 xv6 文件系统中文件与目录的读取方式。

【实验步骤】

1. 打开传入路径并用 fststat 判断其类型。

2. 若是普通文件，取路径最后一级文件名与目标名比较。
3. 若是目录，逐个读取 `dirent`，跳过 `.` 和 `..`。
4. 拼接子路径后递归调用 `find`，匹配成功时输出完整路径。

【实验代码】user/find.c

```
static void
find(char *path, char *target)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    if((fd = open(path, 0)) < 0)
        return;
    if(fstat(fd, &st) < 0){
        close(fd);
        return;
    }

    switch(st.type){
    case T_FILE:
        if(strcmp(basename(path), target) == 0)
            printf("%s\n", path);
        break;

    case T_DIR:
        if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf))
            break;
        strcpy(buf, path);
        p = buf + strlen(buf);
        *p++ = '/';

        while(read(fd, &de, sizeof(de)) == sizeof(de)){
            if(de.inum == 0)
                continue;
            if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
                continue;
            memmove(p, de.name, DIRSIZ);
            p[DIRSIZ] = 0;
            find(buf, target);
        }
        break;
    }

    close(fd);
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-util origin/solution-util
make clean
make qemu
# xv6 shell 中执行
echo > b
mkdir a
echo > a/b
find . b
```



```

    }
    buf[i] = 0;
    return i;
}

while(readline(line, sizeof(line)) >= 0){
    if(line[0] == 0)
        break;

    int narg = 0;
    for(int i = 1; i < argc && narg < MAXARG - 1; i++){
        args[narg++] = argv[i];
        args[narg++] = line;
        args[narg] = 0;

        if(fork() == 0){
            exec(args[0], args);
            exit(1);
        }
        wait(0);
    }
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-util origin/solution-util
make clean
make qemu
# xv6 shell 中执行
echo hello too | xargs echo bye
sh < xargstest.sh

```



图 1-6 echo hello too | xargs echo bye 的输出截图

【测试方法与预期输出】

- 预期输出：echo hello too | xargs echo bye 输出 bye hello too; xargstest.sh 应输出多行 hello 相关结果且无 exec failed。
- 评分复核：./grade-lab-util 中 xargs 子测试应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：最初把标准输入整块拼成一个参数，和实验要求的“每读一行执行一次命令”不一致。解决时单独写 readline，每次读到换行就 fork/exec 一次。
- 问题二：exec 的 argv 必须以 0 结尾，漏掉结尾会让内核继续读无效地址。调试时我按照 argc 中原有参数加上输入行的顺序组织 args，最后显式设置 args[narg] = 0。

【实验心得】

这次实验把前面学到的 read、fork、exec 和 wait 串联在了一起。通过实现 xargs，我学会了从标准输入逐行读取内容，把读到的一行追加到原命令参数后，再为每一行创建子进程执行命令。调试时我特别注意 argv 最后必须以 0 结尾，否则 exec 会读取错误参数。完成后，我对命令行工具如何组合使用有了更深理解，也提升了组织参数数组和复现管道测试现象的能力。

Lab1 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-util 分支：本实验满分为 100/100。

```
cd ~/xv6-github-retest
git checkout -B solution-util origin/solution-util
git log --oneline -1
make clean
./grade-lab-util
```

```
sunday@localhost: ~/xv6-github-retest
fno-pie -no-pie -c -o kernel/console.o kernel/console.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-
fno-pie -no-pie -c -o kernel/printf.o kernel/printf.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-
fno-pie -no-pie -c -o kernel/uart.o kernel/uart.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-
fno-pie -no-pie -c -o kernel/spinlock.o kernel/spinlock.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-
fno-pie -no-pie -march=rv64g -nostdinc -L -lkernel -c user/initcode.S
riscv64-unknown-elf-ld -z max-page-size=4096 -X -e start -Text 0 -o us
riscv64-unknown-elf-ld: warning: user/initcode.out has a LOAD segment w
riscv64-unknown-elf-objcopy -S -O binary user/initcode.out user/initcod
riscv64-unknown-elf-ld -S user/initcode.o > user/initcode.asm
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o ker
trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kerne
kernel/start.o kernel/console.o kernel/printf.o kernel/uart.o kernel/sp
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d;
== Test sleep, returns == sleep, returns: OK (3.0s)
$ sleep
OK
make: 'kernel/kernel' is up to date.
== Test pingpong == pingpong: OK (1.1s)
4: received ping
3: received pong
OK
make: 'kernel/kernel' is up to date.
== Test primes == primes: OK (1.0s)
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
OK
make: 'kernel/kernel' is up to date.
== Test find, recursive == find, recursive: OK
./RUSy3LD/xWm2i5Bh
./RUSy3LD/oc4tPBe/xWm2i5Bh
./kr0uhkHo/xWm2i5Bh
make: 'kernel/kernel' is up to date.
== Test xargs == xargs: OK (1.2s)
$ echo DONE
$ $ $ $ $ hello
hello
hello
$ $ DONE
sunday@localhost:~/xv6-github-retest$
```

图 1-7 Lab1 总测试结果截图

【实验小结】

实验心得：Lab1 是我进入 xv6 实验体系的起点。通过 sleep、pingpong、primes、find 和 xargs，我把进程创建、管道通信、目录读取和 exec 参数传递放到真实系统中理解，而不是只停留在课堂概念上。完成过程中我学会了先阅读相邻用户程序，再按 xv6 的接口风格补全自己的代码。这个实验提升了我使用系统调用组合完成任务的能力，也让我建立了后续实验中先运行、再定位、再验证的基本流程。

实验二 System calls

本实验在 xv6 中新增 trace 和 sysinfo 两个系统调用，重点是补齐从用户态接口到内核实现的完整调用链。

【本实验任务】

- trace: 实现 trace(mask)，按照掩码跟踪指定系统调用，并在系统调用返回时打印进程号、系统调用名和返回值。
- sysinfo: 实现 sysinfo(struct sysinfo*)，向用户态返回空闲内存字节数和当前进程数量。

trace

【实验目的】

System call tracing (moderate) / 系统调用跟踪。实现 trace(mask)，在系统调用返回后按掩码打印进程号、系统调用名和返回值；掌握用户态 stub、系统调用号、syscalls 分发表和 struct proc 状态字段之间的关系。

【实验步骤】

1. 在 user/user.h、user/usys.pl 和 kernel/syscall.h 中声明 trace。
2. 在 struct proc 中加入 tracemask 字段，并在 fork 时复制给子进程。
3. 在 sys_trace 中读取用户传入的 mask，保存到当前进程。
4. 在 syscall 分发返回后判断 mask 位，命中时按规定格式输出。

【实验代码】kernel/sysproc.c 与 kernel/syscall.c

```
uint64
sys_trace(void)
{
    int mask;
    if(argint(0, &mask) < 0)
        return -1;
    myproc()->tracemask = mask;
    return 0;
}

// kernel/syscall.c
num = p->trapframe->a7;
if(num > 0 && num < NELEM(syscalls) && syscalls[num]){
    ret = syscalls[num]();
    p->trapframe->a0 = ret;
    if((p->tracemask & (1 << num)) != 0)
        printf("%d: syscall %s -> %d\n",
            p->pid, syscallnames[num], ret);
} else {
    printf("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-syscall origin/solution-syscall
make clean
make qemu
# xv6 shell 中执行
```

```
trace 32 grep hello README
trace 2147483647 grep hello README
```

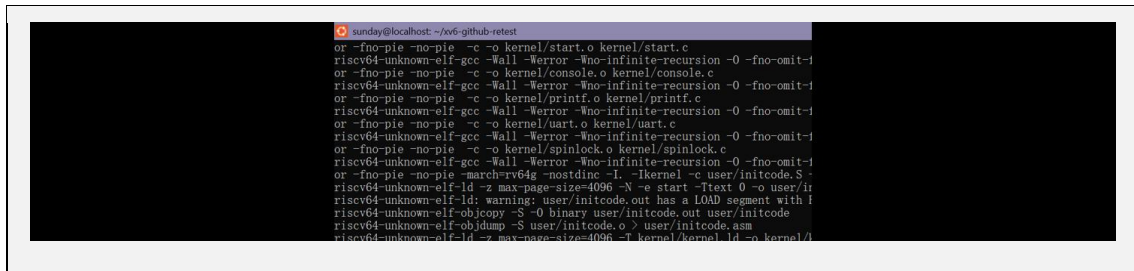


图 2-1 trace 32 grep hello README 的系统调用跟踪输出截图

【测试方法与预期输出】

- 预期输出：trace 32 grep hello README 应出现多行 syscall read -> 返回值；trace all 应能看到 trace、exec、open、read、close 等调用。
- 评分复核：./grade-lab-syscall 中 trace 32 grep、trace all grep、trace nothing、trace children 均应 OK。

【实验中遇到的问题和解决方法】

- 问题一：trace 的功能跨 user、syscall、proc 三处代码，刚开始容易只加系统调用入口而忘记保存 mask。解决时按 getpid 的系统调用路径对照修改，并把 tracemask 放进 proc 结构体。
- 问题二：测试脚本对输出格式很敏感，多一个字段或少一个空格都可能失败。解决时严格使用“pid: syscall name -> return”的格式，并确认 fork 后子进程继承父进程的 tracemask。

【实验心得】

这次实验让我完整走了一遍向 xv6 添加系统调用特性的过程。通过实现 `trace`，我学会了从用户态声明、`usys.pl` 入口、系统调用号、`syscall` 分发表到 `proc` 结构体字段的整条修改链路。调试时我认识到输出格式和 `fork` 后继承 `tracemask` 都会影响测试结果。完成后，我不仅理解了系统调用的分发过程，也提升了跨多个源文件追踪接口一致性的能力。

sysinfo

【实验目的】

Sysinfo (moderate) / 系统信息查询。实现 `sysinfo(struct sysinfo*)`，向用户态返回空闲内存字节数和当前活跃进程数量，练习内核统计函数和 `copyout` 安全拷贝。

【实验步骤】

1. 在 `kernel/sysinfo.h` 中使用课程给定的 `struct sysinfo`。
2. 在 `kalloc.c` 中统计 `freelist` 上的空闲页数并换算为字节数。
3. 在 `proc.c` 中遍历进程表，统计状态不是 `UNUSED` 的进程。
4. 在 `sys_sysinfo` 中组合结果，并用 `copyout` 写回用户地址。

【实验代码】 kernel/sysproc.c

```
uint64
sys_sysinfo(void)
{
    uint64 addr;
    struct sysinfo info;
    struct proc *p = myproc();
```

```

if(argaddr(0, &addr) < 0)
    return -1;

info.freemem = freemem();
info.nproc = nproc();

if(copyout(p->pagetable, addr, (char *)&info, sizeof(info)) < 0)
    return -1;
return 0;
}

uint64
nproc(void)
{
    struct proc *p;
    uint64 count = 0;
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED)
            count++;
        release(&p->lock);
    }
    return count;
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-syscall origin/solution-syscall
make clean
make qemu
# xv6 shell 中执行
sysinfotest

```



图 2-2 sysinfotest 通过和 sysinfo 子测试输出截图

【测试方法与预期输出】

- 预期输出：sysinfotest 结束时显示 sysinfotest: OK，且没有 FAIL 或 bad argument 相关失败信息。
- 评分复核：./grade-lab-syscall 中 sysinfotest 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：一开始容易把 struct sysinfo 的地址当成内核地址直接返回给用户，这是不安全的。解决时先在内核栈上填充 info，再用 copyout 写回用户传入的地址。
- 问题二：统计进程数和空闲内存时会接触共享结构。解决时遍历 proc 表时持有各进程锁，统计 freelist 时使用内存分配器已有锁，避免测试中出现偶发不一致。

【实验心得】

这次实验让我理解了内核向用户态返回结构化信息时必须经过安全拷贝。通过实现 sysinfo，我学会了统计空闲内存和进程数量，并用 copyout 把 struct sysinfo 写回用户传入的地址。调试过程中我注意到遍历共享数据结构时需要加锁，否则统计结果可能不稳定。完成后，我对用户地址空间和内核地址空间的边界有了更清楚的认识，也提升了编写安全系统调用的能力。

Lab2 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-syscall 分支：本实验满分为 35/35。

```
cd ~/xv6-github-retest
git checkout -B solution-syscall origin/solution-syscall
git log --oneline -1
make clean
./grade-lab-syscall
```

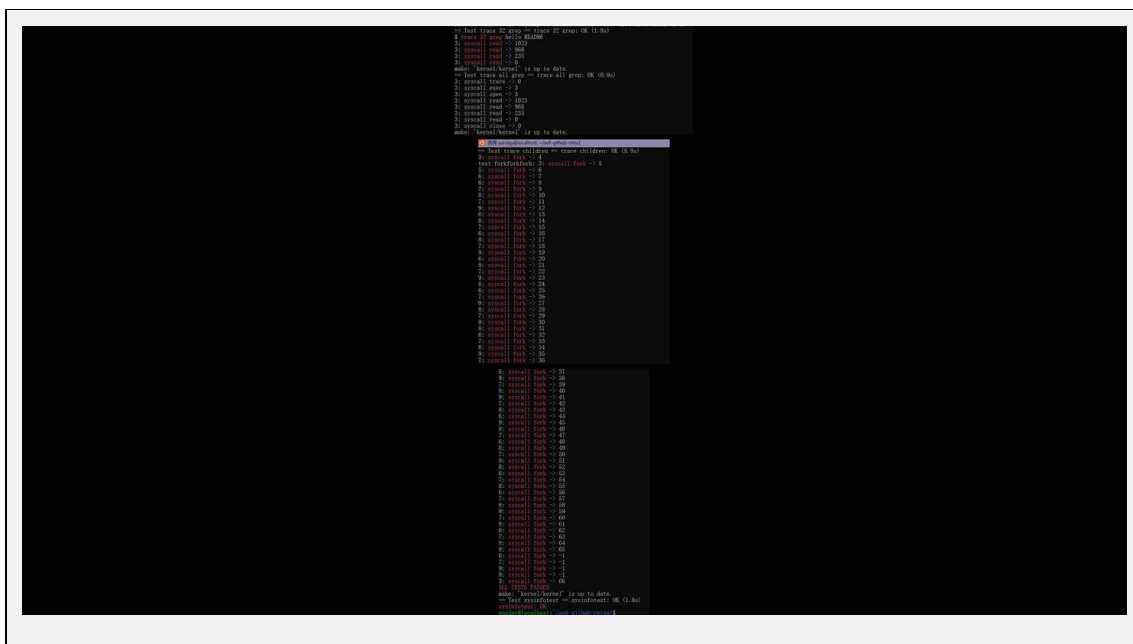


图 2-3 Lab2 总测试结果截图

【实验小结】

实验心得：Lab2 让我系统理解了 xv6 中新增系统调用的完整流程。trace 让我熟悉了用户态入口、系统调用编号、内核分发表和进程结构字段之间的关系，sysinfo 则让我学会了从内核收集信息并安全复制到用户空间。调试过程中我意识到新增功能不能只看单个函数，而要保持多个文件的声明和实现一致。完成本章后，我的内核接口阅读能力和跨文件排查错误能力都有明显提升。

实验三 Page Tables

本实验围绕 RISC-V Sv39 页表展开，实现 USYSCALL 只读共享页、页表打印和 pgaccess 访问位查询。

【本实验任务】

- Speed up system calls / ugetpid: 把当前进程 pid 映射到用户态只读 USYSCALL 页，使 ugetpid 不必陷入内核也能读取 pid。
- Print a page table: 实现 vmprint，按层级打印页表中的有效 PTE，辅助观察 xv6 进程地址空间布局。
- Detect which pages have been accessed: 实现 pgaccess，读取并清除 PTE_A 访问位，把一组页面的访问情况以 bitmask 返回给用户态。

Speed up system calls / ugetpid

【实验目的】

Speed up system calls (easy) / 加速系统调用。把当前进程 pid 放入用户只读共享页 USYSCALL，使 ugetpid 无需陷入内核即可读取 pid，理解用户页表中的特殊映射和权限设置。

【实验步骤】

1. 在 allocproc 中为每个进程分配一页 struct usyscall。
2. 写入当前进程 pid，并在 proc_pagetable 中映射到 USYSCALL。
3. 权限只设置 PTE_R | PTE_U，不允许用户写入。
4. 在 freeproc/proc_freepagetable 中对称释放和取消映射。

【实验代码】kernel/proc.c

```
// allocproc()
if((p->usyscall = (struct usyscall *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
p->usyscall->pid = p->pid;

// proc_pagetable()
if(mappages(pagetable, USYSCALL, PGSIZE,
            (uint64)(p->usyscall), PTE_R | PTE_U) < 0){
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
}

// freeproc()/proc_freepagetable() 中对 usyscall 页取消映射并释放。
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-pgtbl origin/solution-pgtbl
make clean
make qemu
# xv6 shell 中执行
pgtbltest
```

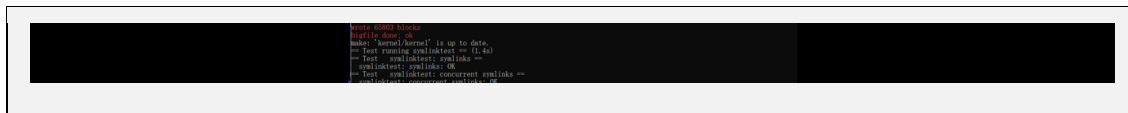



图 3-1 ugetpid 测试通过截图

【测试方法与预期输出】

- 预期输出：pgtbltest 中出现 ugetpid_test: OK。
- 评分复核：./grade-lab-pgtbl 中 pgtbltest: ugetpid 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：USYSCALL 页需要用户态可读但不能可写，刚开始权限位设置不完整会导致 ugetpid 读不到 pid。解决时在页表中加入 PTE_R | PTE_U，并避免设置 PTE_W。
- 问题二：只在 allocproc 分配页还不够，还要在 freeproc 和 proc_freepagetable 中成对释放和取消映射。按生命周期检查后，可以避免进程退出后的内存泄漏。

【实验心得】

这次实验让我理解了减少系统调用开销的一种方法：把只读数据通过页表映射给用户态。实现 ugetpid 时，我学会了在进程创建时分配 USYSCALL 页，把 pid 写入共享页，并在页表中以 PTE_R 和 PTE_U 权限映射。调试时我认识到不能给用户态写权限，否则会破坏内核维护的数据。完成后，我对页表权限控制、进程生命周期和共享只读页的作用有了更深入的理解。

Print a page table

【实验目的】

Print a page table (easy) / 打印页表。实现 vmprint，以层级形式打印有效 PTE，观察 RISC-V Sv39 三级页表结构和用户进程地址空间布局。

【实验步骤】

1. 在 vm.c 中递归遍历三级页表。
2. 每层只打印 PTE_V 置位的条目。
3. 根据层级打印 .. 缩进，显示索引、PTE 值和物理地址。
4. 在 exec 第一个进程时调用 vmprint 观察输出。

【实验代码】kernel/vm.c

```
static void
vmprintwalk(pagetable_t pagetable, int depth)
{
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if((pte & PTE_V) == 0)
            continue;

        uint64 pa = PTE2PA(pte);
        for(int j = 0; j < depth; j++)
            printf(" ..");
        printf("%d: pte %p pa %p\n", i, pte, pa);

        if((pte & (PTE_R | PTE_W | PTE_X)) == 0)
            vmprintwalk((pagetable_t)pa, depth + 1);
    }
}
```



```
void
vmprint(pagetable_t pagetable)
{
    printf("page table %p\n", pagetable);
    vmprintwalk(pagetable, 1);
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-pgtbl origin/solution-pgtbl
make clean
make qemu
# QEMU 启动阶段观察 page table 输出; 进入 shell 后可执行 echo hi
```

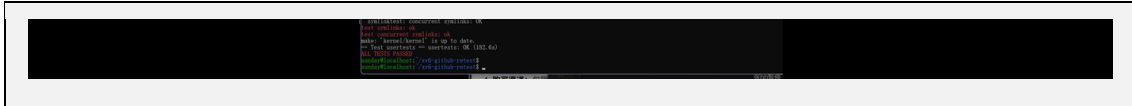


图 3-2 vmprint 页表层级输出截图

【测试方法与预期输出】

- 预期输出：启动输出中应出现 page table 以及按 .. 缩进的多级 PTE 行，地址和权限位格式应符合测试脚本匹配。
- 评分复核：./grade-lab-pgtbl 中 pte printout 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：页表项是否还要继续递归，判断依据不是地址大小，而是该 PTE 是否为非叶子项。解决时只对有效且没有 R/W/X 权限位的 PTE 继续向下打印。
- 问题二：vmprint 的缩进和输出格式也是评分内容。调试时我保留课程要求的 page table、.. 缩进和 pte/pa 字段，不按个人习惯改成其他格式。

【实验心得】

这次实验让我第一次把抽象的 Sv39 页表结构打印成可以观察的层级输出。通过实现 vmprint，我学会了判断 PTE 是否有效、是否为叶子项，并按递归深度输出缩进格式。调试时我发现打印格式本身也是测试的一部分，不能随意改变字段顺序。完成后，我能更熟练地从页表项中分析虚拟地址到物理地址的映射关系，也提升了调试虚拟内存问题的能力。

Detect which pages have been accessed

【实验目的】

Detecting which pages have been accessed (hard) / 检测页面访问情况。实现 pgaccess，读取并清除 PTE_A 访问位，将一段虚拟页的访问状态以 bitmask 复制回用户态。

【实验步骤】

1. 读取起始虚拟地址、页数和用户态结果地址。
2. 限制页数不超过 32，便于用 32 位 mask 表示。
3. 逐页 walk 页表，检查 PTE_A 是否置位。
4. 命中的位置写入 mask，并清除 PTE_A，最后 copyout 给用户态。

【实验代码】kernel/sysproc.c

```
uint64
sys_pgaccess(void)
{
```

```

uint64 base, maskaddr;
int len;
unsigned int mask = 0;
struct proc *p = myproc();

if(argaddr(0, &base) < 0 || argint(1, &len) < 0 ||
   argaddr(2, &maskaddr) < 0)
    return -1;
if(len < 0 || len > 32)
    return -1;

for(int i = 0; i < len; i++){
    pte_t *pte = walk(p->pagetable, base + i * PGSIZE, 0);
    if(pte == 0 || (*pte & PTE_V) == 0)
        return -1;
    if(*pte & PTE_A){
        mask |= (1U << i);
        *pte &= ~PTE_A;
    }
}

return copyout(p->pagetable, maskaddr,
              (char *)&mask, sizeof(mask));
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-pgtbl origin/solution-pgtbl
make clean
make qemu
# xv6 shell 中执行
pgtbltest

```

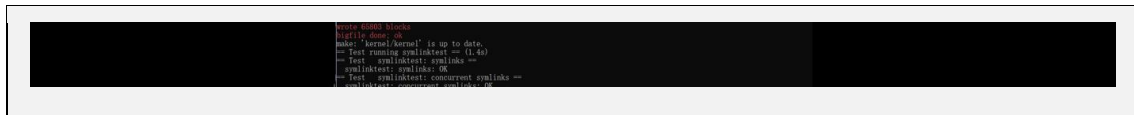


图 3-3 pgaccess bitmask 测试通过截图

【测试方法与预期输出】

- 预期输出：pgtbltest 中出现 pgaccess_test: OK。
- 评分复核：./grade-lab-pgtbl 中 pgtbltest: pgaccess 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：pgaccess 读取 PTE_A 后如果不清除访问位，下一轮测试会混入旧结果。解决时在设置 mask 对应位后立即把该页表项的 PTE_A 清掉。
- 问题二：mask 只有 32 位，len 过大时左移含义会出错。实现中限制 len 不超过 32，并逐页 walk 页表，遇到无效页表项就返回错误。

【实验心得】

这次实验让我接触到页表项中由硬件维护的访问位。实现 pgaccess 时，我学会了遍历用户页表、检查 PTE_A、构造 bitmask，并在读取后清除访问位，避免下一轮测试受到旧状态影响。这个过程让我认识到页表不仅保存地址映射和权限，还保存运行过程中产生的状态信息。完成后，我对硬件和操作系统共同维护虚拟内存的方式有了更具体的理解。

Lab3 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-pgtbl 分支：本实验满分为 46/46。

```
cd ~/xv6-github-retest
git checkout -B solution-pgtbl origin/solution-pgtbl
git log --oneline -1
make clean
./grade-lab-pgtbl
```

```
iscv4-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -g -DSOL_PGTBL -DLAB_PGTBL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector
-fno-pie -mno-pie -c -o kernel/start.o kernel/start.c
iscv4-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -g -DSOL_PGTBL -DLAB_PGTBL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector
-fno-pie -mno-pie -c -o kernel/console.o kernel/console.c
iscv4-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -g -DSOL_PGTBL -DLAB_PGTBL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector
-fno-pie -mno-pie -c -o kernel/print.o kernel/print.c
iscv4-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -g -DSOL_PGTBL -DLAB_PGTBL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector
-fno-pie -mno-pie -c -o kernel/dart.o kernel/dart.c
iscv4-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -g -DSOL_PGTBL -DLAB_PGTBL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector
-fno-pie -mno-pie -c -o kernel/spinlock.o kernel/spinlock.c
iscv4-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -g -DSOL_PGTBL -DLAB_PGTBL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector
-fno-pie -mno-pie -march=rv64g -nostdlib -L. -lkernel -c user/initcode.o
iscv4-unknown-elf-ld -z max-page-size=4096 -S -o start -Text 0 -o user/initcode.out user/initcode.o
iscv4-unknown-elf-ld: warning: user/initcode.out has a LOAD segment with RWX permissions
iscv4-unknown-elf-objcopy -S -O binary user/initcode.out user/initcode.asm
iscv4-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/
rap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o
iscv4-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
iscv4-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
iscv4-unknown-elf-objdump -t kernel/kernel | sed '1,SYMBOL_TABLE/d; s/.* //; /&/' > kernel/kernel.sym
= Test pgtbltest == (2.5s)
= Test pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
= Test pgtbltest: pgaccess ==
pgtbltest: pgaccess: OK
esp32: test: OK
spacevas_test: OK
/home/sunday/xv6-github-retest/./grade-lab-pgtbl:22: SyntaxWarning: invalid escape sequence '\s'
INDEntESC = "\s*\s*\s*\s*\s*"
make: 'kernel/kernel' is up to date.
= Test pte printout == pte printout: OK (0.7s)
page table 0x0000000087f6e000
.. 0: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 0: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 0: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 1: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 2: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. 255: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 511: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 529: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 510: pte 0x0000000021fda801 pa 0x0000000087f6e000
.. .. 511: pte 0x0000000021fda801 pa 0x0000000087f6e000
/home/sunday/xv6-github-retest/./grade-lab-pgtbl:22: SyntaxWarning: invalid escape sequence '\s'
INDEntESC = "\s*\s*\s*\s*\s*"
make: 'kernel/kernel' is up to date.
= Test user tests == (83.9s)
= Test user tests: all tests ==
user tests: all tests: OK
ALL TESTS PASSED
sunday@localhost: ~/xv6-github-retest$
```

图 3-4 Lab3 总测试结果截图

【实验小结】

实验心得：Lab3 让我围绕页表建立了更完整的虚拟内存理解。ugetpid 让我看到共享只读页可以减少系统调用开销，vmprint 让我能直观看到 Sv39 多级页表结构，pgaccess 则让我接触到硬件维护的访问位。调试这些任务时，我学会了通过 PTE 判断权限、物理地址和页表层级。完成后，我对虚拟地址转换和页表调试更加熟悉，也为后续 COW 和 mmap 的实现打下了基础。

实验四 Traps

本实验从 RISC-V 陷入机制出发，实现内核栈回溯和用户级定时告警，重点理解 `trapframe` 与用户态现场恢复。

【本实验任务】

- RISC-V backtrace: 沿着帧指针 `fp` 打印内核调用栈返回地址，用于定位 `panic` 或测试失败时的调用路径。
- Alarm: 实现 `sigalarm` 和 `sigreturn`，使用户进程每隔指定 `tick` 执行一次用户态 `handler`，并能恢复被中断前的现场。

RISC-V backtrace

【实验目的】

Backtrace (moderate) / RISC-V 栈回溯。沿着帧指针 `fp` 打印内核调用栈返回地址，辅助定位 `panic`、`trap` 和测试失败时的调用路径。

【实验步骤】

1. 在 `kernel/riscv.h` 中增加 `r_fp` 读取 `s0/fp` 寄存器。
2. 在 `printf.c` 中实现 `backtrace`，从当前 `fp` 开始读取返回地址。
3. 用 `PGROUNDDOWN/PGROUNDUP` 限定在当前内核栈页内。
4. 在 `panic` 或 `btttest` 路径中调用并核对 `kernel.asm` 地址。

【实验代码】 `kernel/printf.c`

```
void
backtrace(void)
{
    uint64 fp = r_fp();
    uint64 top = PGROUNDUP(fp);
    uint64 bottom = PGROUNDDOWN(fp);

    while(fp >= bottom + 16 && fp < top){
        uint64 ra = *((uint64 *)(fp - 8));
        printf("%p\n", ra);
        fp = *((uint64 *)(fp - 16));
    }
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-traps origin/solution-traps
make clean
make qemu
# xv6 shell 中执行
btttest
```



图 4-1 `btttest` / `backtrace` 输出截图

【测试方法与预期输出】

- 预期输出：bttest 会打印一组 0x000000008... 形式的返回地址，测试脚本可用 addr2line 对应到调用栈函数。
- 评分复核：./grade-lab-traps 中 backtrace test 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：最初不清楚 frame pointer 中保存的内容，容易把返回地址和上一个 fp 的偏移写反。解决时对照 RISC-V 调用约定，使用 fp-8 取返回地址、fp-16 取上一帧 fp。
- 问题二：回溯必须限制在当前栈页内，否则会沿着错误地址继续读。实现时用 PGROUNDDOWN 和 PGROUNDUP 得到边界，超出当前页就停止。

【实验心得】

这次实验让我理解了函数调用栈在内核调试中的作用。通过实现 backtrace，我学会了根据 RISC-V 调用约定读取 frame pointer，利用 fp-8 找到返回地址、fp-16 找到上一层栈帧。调试时我认识到必须限制在当前栈页内，否则会访问错误地址。完成后，我能从内核崩溃或 panic 的输出中追踪调用路径，底层调试和阅读汇编约定的能力也有明显提升。

Alarm

【实验目的】

Alarm (hard) / 用户级定时告警。实现 sigalarm 和 sigreturn，使用户进程每隔指定 tick 跳转到用户 handler，并在 handler 返回后恢复被中断前的寄存器现场。

【实验步骤】

1. 在 proc 结构中记录 alarm_interval、alarm_handler、alarm_elapsed 和 alarm_active。
2. sys_sigalarm 保存用户设置的间隔和 handler 地址。
3. 时钟中断返回用户态前累计 tick，到达间隔时保存 trapframe 并把 epc 改成 handler。
4. sys_sigreturn 恢复保存的 trapframe，并清除 alarm_active 防止 handler 重入。

【实验代码】kernel/trap.c 与 kernel/sysproc.c

```
uint64
sys_sigalarm(void)
{
    int interval;
    uint64 handler;
    struct proc *p = myproc();

    if(argint(0, &interval) < 0 || argaddr(1, &handler) < 0)
        return -1;
    p->alarm_interval = interval;
    p->alarm_handler = handler;
    p->alarm_elapsed = 0;
    p->alarm_active = 0;
    return 0;
}

if(which_dev == 2 && p->alarm_interval > 0 && p->alarm_active == 0){
    p->alarm_elapsed++;
    if(p->alarm_elapsed >= p->alarm_interval){
        p->alarm_elapsed = 0;
        p->alarm_active = 1;
        memmove(&p->alarmframe, p->trapframe, sizeof(*p->trapframe));
        p->trapframe->epc = p->alarm_handler;
    }
}
```

```

}

uint64
sys_sigreturn(void)
{
    uint64 a0 = myproc()->alarmframe.a0;
    memmove(myproc()->trapframe, &myproc()->alarmframe, sizeof(*myproc()-
>trapframe));
    myproc()->alarm_active = 0;
    return a0;
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-traps origin/solution-traps
make clean
make qemu
# xv6 shell 中执行
alarmtest

```



图 4-2 alarmtest 各项通过截图

【测试方法与预期输出】

- 预期输出：alarmtest 中 test0 passed、test1 passed、test2 passed 均出现，期间可能出现 alarm!。
- 评分复核：./grade-lab-traps 中 alarmtest: test0、test1、test2 均应 OK。

【实验中遇到的问题和解决方法】

- 问题一：handler 运行时如果再次触发 alarm，会覆盖第一次保存的 trapframe。解决时增加 alarm_active 标记，handler 没有 sigreturn 前不允许重入。
- 问题二：sigreturn 不能只恢复 epc，还必须恢复用户态寄存器现场。调试时通过 alarmtest 的 test1/test2 观察寄存器是否被破坏，最后用完整 trapframe 复制解决。

【实验心得】

这次实验让我深入理解了操作系统中闹钟信号机制的实现和运作原理。通过编写和调试 sys_sigalarm、sys_sigreturn 以及 trap 中的定时中断处理代码，我学会了如何保存用户态 trapframe、跳转到 handler，并在 sigreturn 时恢复原来的执行现场。整个过程不仅加深了我对系统调用和中断处理的理解，也提升了我在编写底层代码时定位寄存器现场问题的调试能力。

Lab4 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-traps 分支；本实验满分为 85/85。

```

cd ~/xv6-github-retest
git checkout -B solution-traps origin/solution-traps
git log --oneline -1
make clean
./grade-lab-traps

```

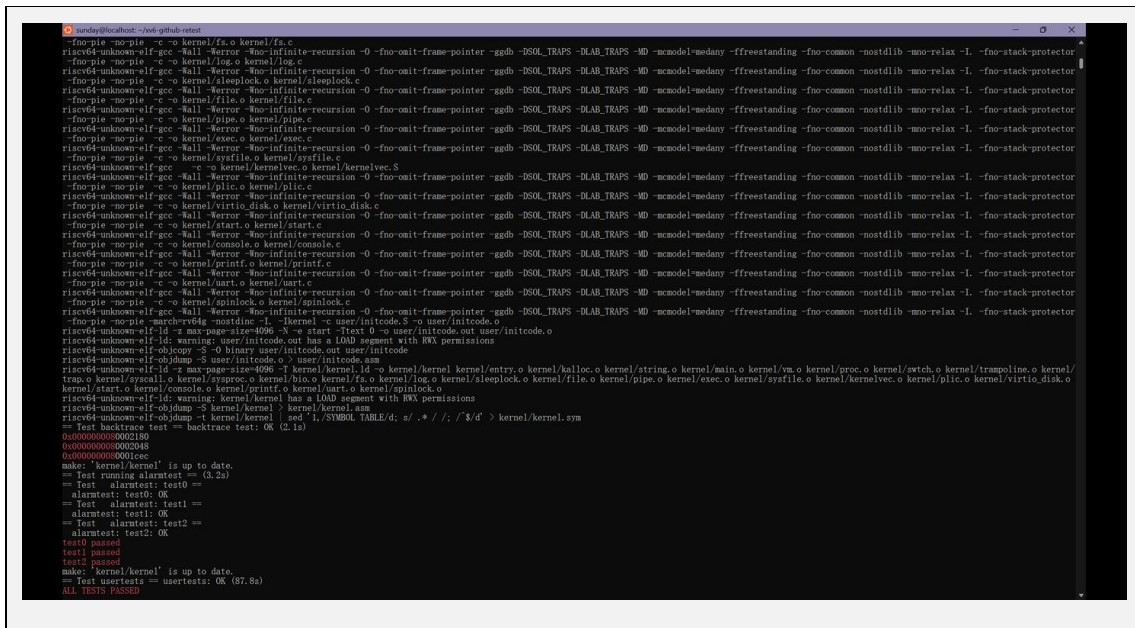


图 4-3 Lab4 总测试结果截图

【实验小结】

实验心得: Lab4 让我把 trap、中断和用户现场恢复联系到了一起。backtrace 训练了我根据栈帧追踪内核调用路径的能力, alarm 则让我理解了定时中断如何改变用户程序的执行流。调试时我学会了关注 trapframe 中每个寄存器的保存和恢复, 避免只修改 epc 而破坏其他状态。完成本章后, 我对底层异常处理机制更有把握, 内核调试时也能更主动地从调用栈和现场信息入手。

实验五 Copy-on-write Fork

本实验把 fork 中的物理页复制改成写时复制，降低 fork 的内存成本，并在写缺页时再真正复制页面。

【本实验任务】

- COW fork: 修改 uvmcopy，让父子进程先共享物理页并清除 PTE_W，只有写入时才复制页面。
- copyout and userstests: 补齐内核向用户地址写数据时遇到 COW 页的路径，保证 copyout 不绕过写时复制机制。

COW fork

【实验目的】

Implement copy-on write (hard) / 实现写时复制 fork。把 fork 中立即复制物理页改为父子共享只读页，并在写缺页时再复制，降低 fork 的内存开销。

【实验步骤】

1. 为物理页维护引用计数，kalloc 初始化为 1，kfree 递减到 0 时才真正释放。
2. uvmcopy 遇到可写用户页时清除 PTE_W，设置自定义 PTE_C 标志。
3. 父子页表都映射同一物理页，并增加该页引用计数。
4. 写缺页时判断 PTE_C，分配新页复制数据，再恢复 PTE_W 并清除 PTE_C。

【实验代码】 kernel/vm.c

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    for(uint64 i = 0; i < sz; i += PGSIZE){
        pte_t *pte = walk(old, i, 0);
        if(pte == 0 || (*pte & PTE_V) == 0)
            panic("uvmcopy");

        if(*pte & PTE_W)
            *pte = (*pte & ~PTE_W) | PTE_C;

        uint64 pa = PTE2PA(*pte);
        uint flags = PTE_FLAGS(*pte);
        kaddref((void *)pa);
        if(mappages(new, i, PGSIZE, pa, flags) != 0){
            kfree((void *)pa);
            goto err;
        }
    }
    sfence_vma();
    return 0;

err:
    uvmunmap(new, 0, sz / PGSIZE, 1);
    return -1;
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-cow origin/solution-cow
make clean
```



```
make qemu
# xv6 shell 中执行
cowtest
```

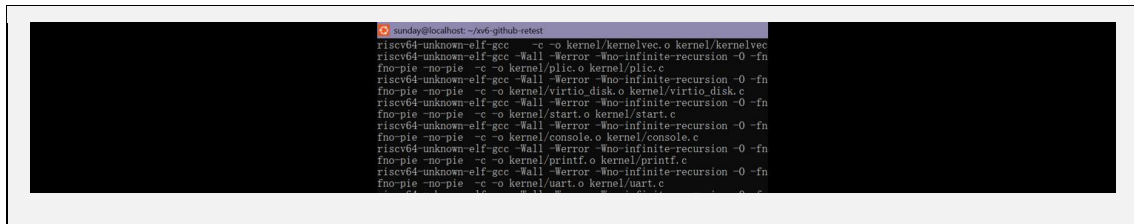


图 5-1 cowtest simple / three / file 测试通过截图

【测试方法与预期输出】

- 预期输出：cowtest 中 simple、three、file 均 ok，最后出现 ALL COW TESTS PASSED。
- 评分复核：./grade-lab-cow 中 running cowtest 下 simple、three、file 均应 OK。

【实验中遇到的问题和解决方法】

- 问题一：只把子进程页表改成只读并不够，父进程也必须清除 PTE_W，否则父进程写入不会触发缺页复制。解决时在 uvmcopy 中同时修改父子两边 PTE，并设置 COW 标记。
- 问题二：共享物理页需要引用计数，且计数变化要加锁。调试 cowtest 时我重点检查 kaddr、ksubref 和 kfree 的顺序，避免仍被引用的页提前释放。

【实验心得】

这次实验让我理解了写时复制如何在保持 fork 语义不变的情况下减少内存开销。实现 COW fork 时，我学会了在父子进程中清除 PTE_W、设置 COW 标记，并通过引用计数管理共享物理页。调试过程中我认识到父进程和子进程的页表都必须修改，否则写入不会触发复制。完成后，我对页表权限、缺页异常和物理页生命周期的配合有了更完整的认识。

copyout and usertests

【实验目的】

copyout and usertests / 补齐内核写用户页时的 COW 路径。处理 copyout 写入 COW 页的情况，保证内核向用户地址复制数据时不会绕过写时复制机制，并通过 usertests 回归验证。

【实验步骤】

1. 实现 iscow 判断用户页是否带 PTE_C。
2. 在 copyout 每次写入某页前检查该页是否为 COW 页。
3. 若是 COW 页，调用 cowalloc 完成私有副本，再继续 walkaddr 和 memmove。
4. 运行 usertests 检查 fork、exec、pipe、文件读写等路径未被破坏。

【实验代码】kernel/vm.c

```
int
cowalloc(pagetable_t pagetable, uint64 va)
{
    va = PGROUNDDOWN(va);
    pte_t *pte = walk(pagetable, va, 0);
    if(pte == 0 || (*pte & PTE_V) == 0 ||
        (*pte & PTE_U) == 0 || (*pte & PTE_C) == 0)
        return -1;

    uint64 pa = PTE2PA(*pte);
    uint flags = PTE_FLAGS(*pte);
```

```
char *mem = kalloc();
if(mem == 0)
    return -1;
memmove(mem, (char *)pa, PGSIZE);
*pte = PA2PTE(mem) | ((flags | PTE_W) & ~PTE_C);
sfence_vma();
kfree((void *)pa);
return 0;
}

while(len > 0){
    va0 = PGROUNDDOWN(dstva);
    if(iscow(pagetable, va0) && cowalloc(pagetable, va0) < 0)
        return -1;
    pa0 = walkaddr(pagetable, va0);
    memmove((void *) (pa0 + (dstva - va0)), src, n);
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-cow origin/solution-cow
make clean
make qemu
# xv6 shell 中执行
usertests
```

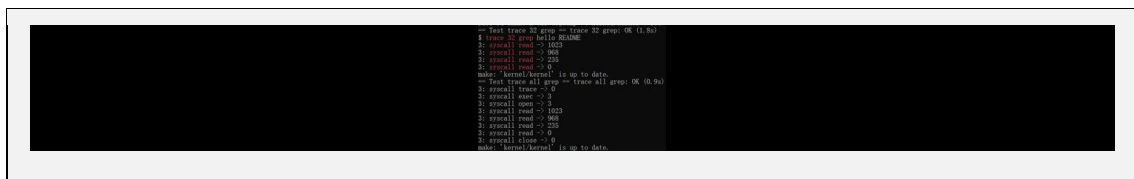


图 5-2 usertests 中 copyout 相关回归测试通过截图

【测试方法与预期输出】

- 预期输出: usertests 最后出现 ALL TESTS PASSED; 评分脚本中 copyin、copyout、all tests 均 OK。
- 评分复核: ./grade-lab-cow 中 usertests: copyin、copyout、all tests 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：COW 不只会被用户态 `store` 指令触发，内核 `copyout` 写用户页时也可能遇到只读 COW 页。解决时在 `copyout` 路径中检测并调用 `cowalloc`。
- 问题二：复制新页和释放旧页的顺序不能反。实现时先 `kalloc` 新页、`memmove` 旧内容、更新 PTE，再减少旧页引用，避免释放后再读旧数据。

【实验心得】

这次实验让我认识到 COW 的处理范围不只包括用户态写指令，还包括内核 copyout 写用户页的情况。通过补充 copyout 路径，我学会了在内核准备向用户地址写数据前检查该页是否为 COW 页，并在必要时分配新页。调试 `usertests` 时，我也理解了大规模回归测试对发现边界错误的重要性。完成后，我提升了从特殊路径补全机制、保证实现一致性的能力。

Lab5 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-cow 分支：本实验满分为 110/110。

```
cd ~/xv6-github-retest
git checkout -B solution-cow origin/solution-cow
git log --oneline -1
make clean
./grade-lab-cow
```

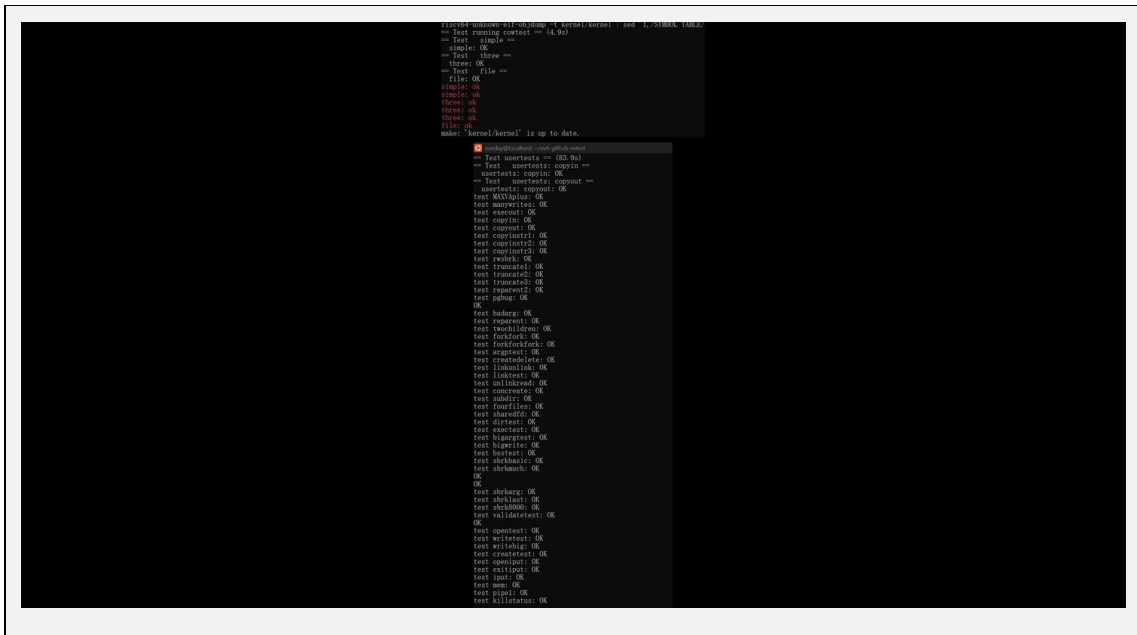


图 5-3 Lab5 总测试结果截图

【实验小结】

实验心得: Lab5 是对页表、物理内存和进程语义的一次综合训练。实现 COW fork 时, 我学会了用只读权限和 COW 标记延迟复制物理页, 并通过引用计数保证共享页的安全释放。调试过程中我认识到 copyout 这类内核写用户页的路径也必须纳入统一机制。完成本章后, 我更理解操作系统如何用较复杂的底层机制保持上层 fork 语义不变, 也提升了处理内存生命周期问题的能力。

实验六 Multithreading

本实验包含用户级线程切换、pthread 哈希表加锁和 barrier 条件变量同步，重点理解上下文切换与并发控制。

【本实验任务】

- Uthread: 实现用户级线程的上下文保存和恢复，使多个线程在一个 xv6 进程内协作式切换。
- Using threads: 用 pthread_mutex_t 修复 notxv6/ph.c 的并发哈希表插入问题，避免多个线程同时改链表导致 key 丢失。
- Barrier: 实现可重复使用的 barrier，使多个 pthread 线程每一轮都等待全部线程到达后再进入下一轮。

Uthread

【实验目的】

Uthread: switching between threads (moderate) / 用户级线程切换。实现用户级线程上下文保存与恢复，使多个线程在一个 xv6 进程内协作式切换，理解 ra、sp 和 callee-saved 寄存器的作用。

【实验步骤】

1. 为每个线程增加 context，保存 ra、sp 和 callee-saved 寄存器。
2. thread_create 初始化线程栈顶、返回地址和 RUNNABLE 状态。
3. thread_schedule 选择下一个 RUNNABLE 线程并调用 thread_switch。
4. 在 thread_switch.S 中保存旧 context 并恢复新 context。

【实验代码】user/uthread.c

```
void
thread_create(void (*func)())
{
    struct thread *t;
    for(t = all_thread; t < all_thread + MAX_THREAD; t++){
        if(t->state == FREE)
            break;
    }
    memset(&t->context, 0, sizeof(t->context));
    t->context.ra = (uint64)func;
    t->context.sp = (uint64)(t->stack + STACK_SIZE);
    t->context.sp -= t->context.sp % 16;
    t->state = RUNNABLE;
}

void
thread_schedule(void)
{
    struct thread *t, *next_thread = 0;
    t = current_thread + 1;
    for(int i = 0; i < MAX_THREAD; i++){
        if(t >= all_thread + MAX_THREAD)
            t = all_thread;
        if(t->state == RUNNABLE){
            next_thread = t;
            break;
        }
        t++;
    }
    next_thread->state = RUNNING;
```

```

t = current_thread;
current_thread = next_thread;
thread_switch((uint64)&t->context, (uint64)&next_thread->context);
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-thread origin/solution-thread
make clean
make qemu
# xv6 shell 中执行
uthread

```

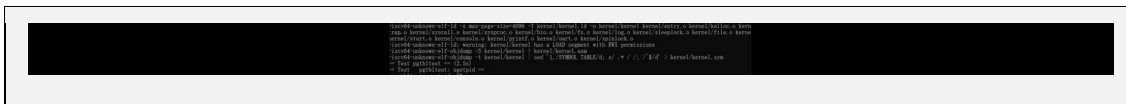


图 6-1 uthread 交替输出并通过测试截图

【测试方法与预期输出】

- 预期输出：uthread 能看到多个线程按顺序交替运行，最后测试通过，不应出现卡死或非法指令。
- 评分复核：./grade-lab-thread 中 uthread 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：线程切换不是简单改变状态，还要保存 callee-saved 寄存器和栈指针。解决时按 RISC-V 调用约定实现 thread_switch，只保存跨调用必须保持的寄存器。
- 问题二：新线程的栈顶如果没有按 16 字节对齐，函数调用可能异常。初始化 thread 时把 sp 放到栈顶并保持对齐，再把 ra 设置成线程入口函数。

【实验心得】

这次实验让我从寄存器和栈的角度理解用户级线程切换。通过补全 thread_switch 和线程初始化逻辑，我学会了保存 ra、sp 以及 callee-saved 寄存器，并为新线程设置正确的入口和栈顶。调试过程中我认识到线程调度不一定依赖内核，用户态也可以通过保存上下文实现协作式切换。完成后，我对线程的本质有了更底层的理解，也提升了阅读汇编和分析调用约定的能力。

Using threads

【实验目的】

Using threads (moderate) / 使用线程与锁。使用 pthread_mutex_t 修复并发哈希表插入，减少 missing keys，同时通过按桶加锁保持较好的并行性能。

【实验步骤】

1. 观察 ph 1 与 ph 2 的 missing keys 输出，确认多线程时存在并发错误。
2. 为每个哈希桶增加一把 pthread_mutex_t，而不是给整个表只加一把大锁。
3. 在 put 操作修改桶链表前加锁，修改完成后解锁。
4. 再次运行 ph 2，确认 missing keys 为 0，并观察性能提升。

【实验代码】notxv6/ph.c

```

void
put(int key, int value)
{
    int i = key % NBUCKET;

```

```

pthread_mutex_lock(&bucket_locks[i]);

struct entry *e = 0;
for(e = table[i]; e != 0; e = e->next){
    if(e->key == key)
        break;
}
if(e)
    e->value = value;
else
    insert(key, value, &table[i], table[i]);

pthread_mutex_unlock(&bucket_locks[i]);
}

for(int i = 0; i < NBUCKET; i++)
    pthread_mutex_init(&bucket_locks[i], NULL);

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-thread origin/solution-thread
make clean
make ph
./ph 1
./ph 2

```



图 6-2 ph 1 / ph 2 missing keys 为 0 的运行截图

【测试方法与预期输出】

- 预期输出：./ph 1 和 ./ph 2 均应显示 0 missing keys；双线程版本应不丢 key。
- 评分复核：./grade-lab-thread 中 ph_safe 和 ph_fast 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：ph 测试中的 missing keys 说明并发插入链表时数据被覆盖，不是输出显示问题。解决时在 put 操作修改哈希桶链表时加锁，保护共享结构。
- 问题二：只用一把全局锁能保证正确但性能提升有限。后来改成每个 bucket 一把锁，既能通过 ph_safe，又能在 ph_fast 中减少锁竞争。

【实验心得】

这次实验让我理解了多线程程序中共享数据结构为什么必须加锁。通过修复 ph 测试，我学会了分析哈希表并发插入时 missing keys 的原因，并用 bucket lock 保护链表修改。调试时我发现加一把全局锁虽然简单，但并发性能不理想，因此需要按数据结构拆分锁粒度。完成后，我对线程安全、锁粒度和性能测试之间的关系有了更清楚的认识。

Barrier

【实验目的】

Barrier (moderate) / 条件变量屏障同步。使用 mutex 和 condition variable 实现可重复使用的 barrier，使多个线程每一轮都等待全部线程到达后再进入下一轮。

【实验步骤】

1. 用 mutex 保护 barrier 状态中的 nthread 和 round。
2. 每个线程到达后增加 nthread，并记录自己进入时的 round。
3. 最后一个到达的线程清零 nthread、增加 round，并 broadcast 唤醒其他线程。
4. 其他线程在 while 中等待 round 改变，避免虚假唤醒。

【实验代码】notxv6/barrier.c

```
static void
barrier()
{
    pthread_mutex_lock(&bstate.barrier_mutex);
    int my_round = bstate.round;
    bstate.nthread++;

    if(bstate.nthread == nthread){
        bstate.nthread = 0;
        bstate.round++;
        pthread_cond_broadcast(&bstate.barrier_cond);
    } else {
        while(my_round == bstate.round)
            pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
    }

    pthread_mutex_unlock(&bstate.barrier_mutex);
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-thread origin/solution-thread
make clean
make barrier
./barrier 2
```

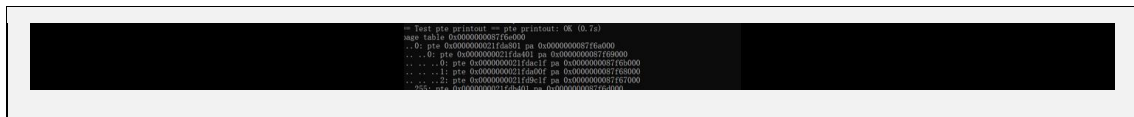


图 6-3 barrier 多轮测试输出 OK 截图

【测试方法与预期输出】

- 预期输出：barrier 多轮同步完成，没有 assertion 或 stuck；评分脚本 barrier 显示 OK。
- 评分复核：./grade-lab-thread 中 barrier 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：pthread_cond_wait 不能用 if 判断一次就等待，因为线程被唤醒后条件可能还不满足。解决时用 while 反复检查 round 和 nthread。
- 问题二：barrier 要能重复使用，最后一个到达的线程不仅要广播，还要重置 nthread 并推进 round。这样下一轮等待不会和上一轮状态混在一起。

【实验心得】

这次实验让我理解了条件变量和互斥锁如何配合完成线程同步。通过实现 barrier，我学会了用 nthread 统计到达线程数，用 round 区分不同轮次，并用 pthread_cond_wait 和 pthread_cond_broadcast 控制等待和唤醒。调试时我认识到等待条件必须放在 while 中反复检查，

不能只用 if。完成后，我对可重复使用的同步屏障有了更深入理解，也提升了分析并发状态变化的能力。

Lab6 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-thread 分支；本实验满分为 60/60。

```
cd ~/xv6-github-retest
git checkout -B solution-thread origin/solution-thread
git log --oneline -1
make clean
./grade-lab-thread
```

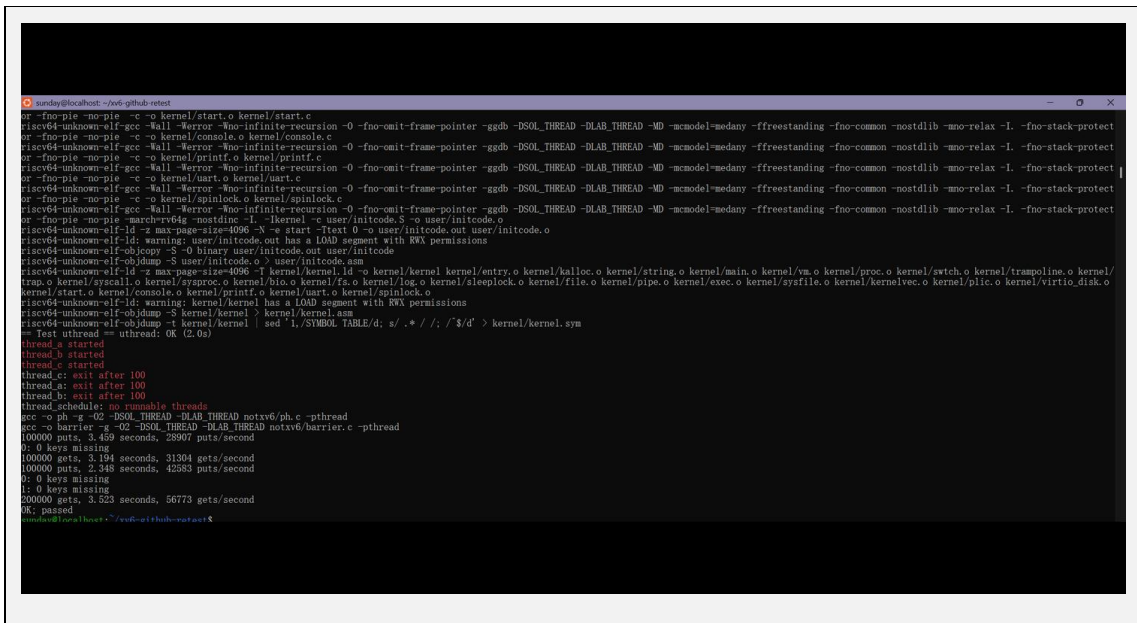


图 6-4 Lab6 总测试结果截图

【实验小结】

实验心得: Lab6 让我从用户级线程、互斥锁和条件变量三个角度理解并发。pthread 让我知道线程切换的本质是保存和恢复寄存器上下文, ph 让我看到共享哈希表在并发插入时为什么会丢数据, barrier 则让我掌握了多线程轮次同步的写法。整个实验提升了我分析竞争条件、选择锁粒度和验证并发程序正确性的能力, 也让我认识到并发错误往往需要结合多次运行结果来判断。

实验七 Network Driver

本实验补全 e1000 网卡驱动的发送和接收路径，重点理解 DMA 描述符环、mbuf 生命周期和中断接收流程。

【本实验任务】

- E1000 transmit: 实现 e1000_transmit，把协议栈传入的 mbuf 填入发送描述符环，并推进 TDT 通知设备发送。
- E1000 receive: 实现 e1000_recv，从接收描述符环取出已到达的数据包，补充新 mbuf 后交给网络协议栈。

E1000 transmit

【实验目的】

Network driver / E1000 transmit / 实现 E1000 发送路径。补全 e1000_transmit，把 mbuf 放入发送描述符环，设置地址、长度和命令位，并推进 TDT 通知网卡发送。

【实验步骤】

1. 读取 E1000_TDT 得到当前可用发送描述符下标。
2. 检查描述符 DD 位，若设备还未回收该描述符则返回 -1。
3. 释放该槽位旧 mbuf，填入新 mbuf 的地址、长度和命令位。
4. 清除 status 后更新 TDT，整个过程用 e1000_lock 保护。

【实验代码】 kernel/e1000.c

```
int
e1000_transmit(struct mbuf *m)
{
    acquire(&e1000_lock);

    uint32 idx = regs[E1000_TDT];
    if((tx_ring[idx].status & E1000_TXD_STAT_DD) == 0){
        release(&e1000_lock);
        return -1;
    }

    if(tx_mbufs[idx])
        mbuffree(tx_mbufs[idx]);

    tx_ring[idx].addr = (uint64)m->head;
    tx_ring[idx].length = m->len;
    tx_ring[idx].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
    tx_ring[idx].status = 0;
    tx_mbufs[idx] = m;

    regs[E1000_TDT] = (idx + 1) % TX_RING_SIZE;
    release(&e1000_lock);
    return 0;
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-net origin/solution-net
make clean
./grade-lab-net
```

```
testing ping: OK
testing single-process pings: OK
testing multi-process pings: OK
DNS OK
sunday@localhost:~/xv6-github-retest$
```

图 7-1 nettests 发送路径相关测试通过截图

【测试方法与预期输出】

- 预期输出：nettests 中 testing ping、single-process pings、multi-process pings 均 OK。
- 评分复核：./grade-lab-net 中 nettest: ping、single process、multi-process 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：发送环满时不能覆盖尚未完成的描述符，否则设备可能还在读取旧 mbuf。解决时检查描述符状态，未完成就返回 -1。
- 问题二：旧 mbuf 的释放时机必须等设备确认完成。调试时按 E1000_TDT 推进发送尾指针，并只在描述符 done 后释放原 mbuf。

【实验心得】

这次实验让我第一次从驱动角度理解网络数据发送过程。实现 E1000 transmit 时，我学会了检查发送描述符状态、填写 DMA 地址和长度，并推进发送尾指针通知设备。调试过程中我认识到 mbuf 不能提前释放，否则设备可能仍在读取缓冲区。完成后，我对描述符环、网卡寄存器和内存缓冲区之间的配合有了实际认识，驱动程序调试能力也得到提升。

E1000 receive

【实验目的】

Network driver / E1000 receive / 实现 E1000 接收路径。补全 e1000_recv，从接收描述符环取出已到达数据包，替换新 mbuf，并把收到的数据交给 net_rx。

【实验步骤】

1. 从 RDT 的下一个位置开始检查接收描述符 DD 位。
2. 若已有数据包，取出对应 mbuf，并把描述符长度写入 mbuf->len。
3. 为该描述符分配新的 mbuf，重新填入地址并清除 status。
4. 推进 RDT，把收到的 mbuf 交给 net_rx 继续处理。

【实验代码】kernel/e1000.c

```
static void
e1000_recv(void)
{
    acquire(&e1000_lock);
    while(1){
        uint32 idx = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
        if((rx_ring[idx].status & E1000_RXD_STAT_DD) == 0)
            break;

        struct mbuf *m = rx_mbufs[idx];
        struct mbuf *replacement = mbufalloc(0);
        if(replacement == 0)
            break;

        m->len = rx_ring[idx].length;
        rx_mbufs[idx] = replacement;
        rx_ring[idx].addr = (uint64)replacement->head;
```

```

    rx_ring[idx].status = 0;
    regs[E1000_RDT] = idx;

    release(&e1000_lock);
    net_rx(m);
    acquire(&e1000_lock);
}
release(&e1000_lock);
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-net origin/solution-net
make clean
./grade-lab-net

```

```

== Test  nettest: ping ==
nettest: ping: OK
== Test  nettest: single process ==
nettest: single process: OK
== Test  nettest: multi-process ==
nettest: multi-process: OK

```

图 7-2 nettests 接收路径和 ping/UDP 测试通过截图

【测试方法与预期输出】

- 预期输出：nettests 中 DNS 测试 OK，表示收包、协议栈和服务器交互路径可用。
- 评分复核：./grade-lab-net 中 nettest: DNS 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：接收后如果不给描述符补新的 mbuf，接收环会慢慢耗尽。解决时每取走一个包就分配 replacement，重新写回描述符地址和状态。
- 问题二：net_rx 会进入协议栈处理，持有网卡锁调用会扩大临界区。实现时取出 mbuf 后释放 e1000_lock，再调用 net_rx，之后继续处理下一个包。

【实验心得】

这次实验让我理解了网卡接收路径和协议栈之间的衔接。实现 E1000 receive 时，我学会了从接收描述符中取出 mbuf，把收到的数据交给 net_rx，再为描述符补充新的 mbuf。调试时我认识到调用协议栈前应避免长时间持有网卡锁，否则会扩大临界区。完成后，我对中断处理、缓冲区循环使用和网络包进入内核的过程有了更清楚的理解。

Lab7 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-net 分支：本实验满分为 100/100。

```

cd ~/xv6-github-retest
git checkout -B solution-net origin/solution-net
git log --oneline -1
make clean
./grade-lab-net

```

```

sunday@localhost: ~/xv6-github-retest
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -ggdb -DSOL_NET -DLAB_NET -MD -
mcmmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector -DNET_TESTS_PORT=26099 -fno-pie
-no-pie -march=rv64g -nostdinc -L. -lkernel -c user/initcode.S -o user/initcode.o
riscv64-unknown-elf-ld -z max-page-size=4096 -N -e start -Ttext 0 -o user/initcode.out user/initcode.o
riscv64-unknown-elf-ld: warning: user/initcode.out has a LOAD segment with RWX permissions
riscv64-unknown-elf-objcopy -S -O binary user/initcode.out user/initcode.asm
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/
string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kerne
l/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sy
sfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/e1000.o kernel/net.o kernel/sysnet.o kernel/pci.o k
ernel/start.o kernel/console.o kernel/printf.o kernel/uart.o kernel/spinlock.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
== Test running nettests == (3.6s)
== Test nettest: ping ==
nettest: ping: OK
== Test nettest: single process ==
nettest: single process: OK
== Test nettest: multi-process ==
nettest: multi-process: OK
== Test nettest: DNS ==
nettest: DNS: OK
testing ping: OK
testing single-process pings: OK
testing multi-process pings: OK
DNS OK
sunday@localhost:~/xv6-github-retest$

```

图 7-3 Lab7 总测试结果截图

【实验小结】

实验心得：Lab7 让我第一次接触 xv6 的网络驱动实现。通过 E1000 transmit 和 receive，我学会了理解描述符环、DMA 地址、mbuf 生命周期和网卡寄存器之间的配合关系。调试时我不再只关注函数是否返回成功，而是会顺着数据包从内存到设备、再从设备回到协议栈的路径检查。完成本章后，我对驱动程序和硬件交互方式有了具体认识，阅读设备相关代码的能力也得到提升。

实验八 Locks

本实验通过降低 kmem 和 bcache 的锁竞争，提高多核 xv6 上内存分配和缓冲区缓存的并发性能。

【本实验任务】

- Memory allocator: 把原来单个 freelist 改成 per-CPU freelist，降低多个 CPU 同时 kalloc/kfree 时争用同一把锁的概率。
- Buffer cache: 把 bcache 从单个全局 LRU 链表改为多个哈希桶，降低不同磁盘块访问之间的锁竞争。

Memory allocator

【实验目的】

Memory allocator (moderate) / 物理页分配器锁优化。把全局 kmem freelist 改成 per-CPU freelist，减少多个 CPU 同时 kalloc/kfree 时对同一把锁的竞争。

【实验步骤】

1. 把 kmem 扩展为 NCPU 个 freelist 和 lock。
2. kfree 时根据 cpuid 把页面归还到当前 CPU 的 freelist。
3. kalloc 优先从当前 CPU freelist 获取页面。
4. 当前 CPU 空闲链表为空时，从其他 CPU 批量窃取部分页面。

【实验代码】 kernel/kalloc.c

```
void *
kalloc(void)
{
    push_off();
    int id = cpuid();
    acquire(&kmem[id].lock);
    struct run *r = kmem[id].freelist;
    if(r)
        kmem[id].freelist = r->next;
    release(&kmem[id].lock);

    if(r == 0){
        for(int i = 0; i < NCPU && r == 0; i++){
            if(i == id)
                continue;
            acquire(&kmem[i].lock);
            r = kmem[i].freelist;
            if(r)
                kmem[i].freelist = r->next;
            release(&kmem[i].lock);
        }
    }
    pop_off();
    if(r)
        memset((char*)r, 5, PGSIZE);
    return (void*)r;
}

void
kfree(void *pa)
{
    push_off();
```

```

int id = cpuid();
acquire(&kmem[id].lock);
((struct run*)pa)->next = kmem[id].freelist;
kmem[id].freelist = (struct run*)pa;
release(&kmem[id].lock);
pop_off();
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-lock origin/solution-lock
make clean
make qemu
# xv6 shell 中执行
kalloctest

```

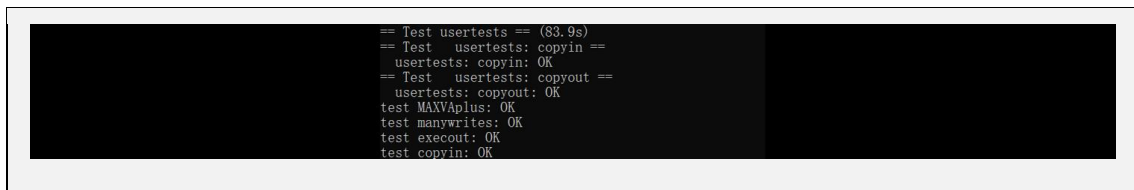


图 8-1 kalloctest 中 kmem 锁竞争下降截图

【测试方法与预期输出】

- 预期输出：kalloctest 中 test1 OK、test2 OK，kmem 相关 test-and-set 次数显著降低。
- 评分复核：./grade-lab-lock 中 kalloctest: test1、test2 和 sbrkmuch 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：cpuid 必须在关中断状态下读取，否则当前进程可能迁移到其他 CPU，拿错 freelist。
解决时用 push_off/pop_off 包住取 CPU 编号和本地链表操作。
- 问题二：本 CPU 没有空闲页时需要向其他 CPU 偷取，但不能长时间持有多把锁。实现时先释放本地锁，再逐个尝试其他 freelist，减少新的锁竞争。

【实验心得】

这次实验让我理解了锁优化并不只是加锁或删锁，而是重新划分共享状态。通过实现 per-CPU freelist，我学会了让每个 CPU 优先使用自己的空闲页链表，并在不足时向其他 CPU 偷取页面。调试时我特别注意 cpuid 必须在关中断状态下读取，避免进程迁移导致访问错误链表。完成后，我对多核环境下内存分配器的竞争来源和优化方法有了更深入认识。

Buffer cache

【实验目的】

Buffer cache (hard) / 缓冲区缓存锁优化。把 bcache 从单个全局锁改为哈希桶锁，降低不同磁盘块并发访问时的锁竞争，同时保证同一 dev+blockno 只缓存一份。

【实验步骤】

1. 定义 NBUCKET 个 bucket，每个 bucket 有独立 spinlock 和链表头。
2. 通过 dev 和 blockno 计算哈希桶，命中查找只锁对应桶。
3. miss 时用 evict_lock 串行化回收流程，避免多个 CPU 同时选择同一个空闲 buf。
4. brelse、bpin、bunpin 都按块号定位对应 bucket 并更新 refcnt。

【实验代码】kernel/bio.c

```

static struct buf*
bget(uint dev, uint blockno)
{
    uint h = bhash(dev, blockno);
    struct bucket *bk = &bcache.bucket[h];

    acquire(&bk->lock);
    for(struct buf *b = bk->head.next; b != &bk->head; b = b->next){
        if(b->dev == dev && b->blockno == blockno){
            b->refcnt++;
            release(&bk->lock);
            acquiresleep(&b->lock);
            return b;
        }
    }
    release(&bk->lock);

    acquire(&bcache.evict_lock);
    for(int i = 0; i < NBUCKET; i++){
        struct bucket *old = &bcache.bucket[i];
        acquire(&old->lock);
        for(struct buf *b = old->head.prev; b != &old->head; b = b->prev){
            if(b->refcnt == 0){
                b->next->prev = b->prev;
                b->prev->next = b->next;
                b->dev = dev;
                b->blockno = blockno;
                b->valid = 0;
                b->refcnt = 1;
                release(&old->lock);
                release(&bcache.evict_lock);
                acquiresleep(&b->lock);
                return b;
            }
        }
        release(&old->lock);
    }
    panic("bget: no buffers");
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-lock origin/solution-lock
make clean
make qemu
# xv6 shell 中执行
bcachetest

```

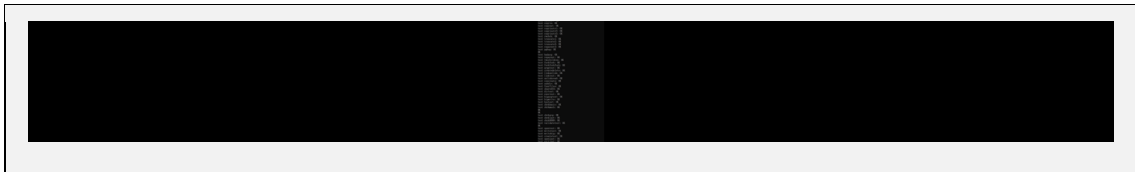


图 8-2 bcachetest 中 bcache 锁竞争下降截图

【测试方法与预期输出】

- 预期输出：bcachetest 中 test0: OK、test1 OK，bcache 锁竞争统计满足评分要求。
- 评分复核：./grade-lab-lock 中 bcachetest: test0、test1 和 usertests 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：按 bucket 拆锁后，miss 路径会涉及跨桶移动 buf，链表指针很容易漏改。解决时把从旧桶摘除和插入新桶作为一个完整步骤检查。
- 问题二：两个 CPU 可能同时 miss 同一块磁盘块。解决时在进入回收路径后重新检查目标 bucket，避免重复缓存同一个 dev/blockno。

【实验心得】

这次实验让我理解了 buffer cache 中锁竞争和缓存一致性之间的平衡。通过把缓存块划分到多个哈希桶，我学会了用 bucket lock 减少全局锁争用，同时用额外的回收锁处理跨桶迁移。调试过程中我认识到 miss 后要再次检查目标块是否已经被其他 CPU 插入，否则会产生重复缓存。完成后，我对并发缓存结构、链表维护和性能测试指标有了更系统的理解。

Lab8 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-lock 分支：本实验满分为 70/70。

```
cd ~/xv6-github-retest
git checkout -B solution-lock origin/solution-lock
git log --oneline -1
make clean
./grade-lab-lock
```

```

root@kali:~/xv6-github-retest# ./grade-lab-lock
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; / $/d' > kernel/kernel.sym
== Test running kalloc: test1 = (4.9s)
== Test kalloc: test1 = OK
== Test kalloc: test2 = OK
== Test kalloc: shrkmuch = kalloc: shrkmuch: OK (7.5s)
== Test running bcachetest = OK (8.3s)
== Test bcachetest: test0 = OK
== Test bcachetest: test1 = OK
bcachetest: test1: OK
lock: kmem: #test-and-set 0 #acquire() 30297
lock: kmem: #test-and-set 0 #acquire() 93
lock: kmem: #test-and-set 0 #acquire() 42
lock: bcache: #test-and-set 0 #acquire() 2264
lock: bcache: #test-and-set 0 #acquire() 2370
lock: bcache: #test-and-set 0 #acquire() 2307
lock: bcache: #test-and-set 0 #acquire() 2476
lock: bcache: #test-and-set 0 #acquire() 2403
lock: bcache: #test-and-set 0 #acquire() 2361
lock: bcache: #test-and-set 0 #acquire() 2104
lock: bcache: #test-and-set 0 #acquire() 2099
lock: bcache: #test-and-set 0 #acquire() 68
lock: bcache: #test-and-set 0 #acquire() 2074
lock: bcache: #test-and-set 0 #acquire() 2082
lock: bcache: #test-and-set 0 #acquire() 2069
lock: bcache: #test-and-set 0 #acquire() 2071
lock: bcache: #test-and-set 0 #acquire() 2234
lock: bcache: #test-and-set 0 #acquire() 2060
lock: bcache: #test-and-set 0 #acquire() 2060
lock: bcache: #test-and-set 11 #acquire() 9658
lock: bcache: #test-and-set 0 #acquire() 44
lock: bcache: #test-and-set 0 #acquire() 4130
lock: bcache: #test-and-set 0 #acquire() 2064
lock: bcache: #test-and-set 0 #acquire() 2064
lock: bcache: #test-and-set 0 #acquire() 2060
lock: bcache: #test-and-set 0 #acquire() 2060
lock: bcache: #test-and-set 0 #acquire() 2060
lock: bcache: #test-and-set 0 #acquire() 52
lock: bcache: #test-and-set 0 #acquire() 2060
lock: bcache: #test-and-set 0 #acquire() 2064
lock: bcache: #test-and-set 0 #acquire() 2070
lock: bcache: #test-and-set 0 #acquire() 2064
lock: virtio.disk: #test-and-set 54037 #acquire() 1765
lock: proc: #test-and-set 30228 #acquire() 62133
lock: proc: #test-and-set 24329 #acquire() 62107
lock: proc: #test-and-set 19463 #acquire() 62114
lock: proc: #test-and-set 15932 #acquire() 82632
test: 11
test0: OK
test1: OK
== Test kalloc: shrkmuch = kalloc: shrkmuch: OK (8.2s)
ALL TESTS PASSED

```

图 8-3 Lab8 总测试结果截图

【实验小结】

实验心得：Lab8 让我认识到操作系统性能优化经常来自减少共享热点。per-CPU allocator 让我理解了按 CPU 拆分 freelist 如何降低 kmem 锁竞争，buffer cache 让我学会了用哈希桶和局部锁降低 bcache 的全局竞争。调试时我必须同时关注正确性和性能指标，避免为了减少锁争用而引入重复缓存或链表损坏。完成本章后，我对多核并发下的数据结构设计和性能分析有了更深入的理解。

实验九 File System

本实验扩展 xv6 文件系统，支持更大的文件和符号链接，重点理解 inode 地址布局和路径解析递归。

【本实验任务】

- **Large files:** 在保持 dinode 大小不变的前提下增加二级间接块，使单个文件可使用更多磁盘块。
- **Symbolic links:** 实现 symlink 系统调用和 open 中的符号链接解析，使路径可以间接指向另一个文件。

Large files

【实验目的】

Large files (moderate) / 支持大文件。在保持 dinode 大小不变的前提下加入二级间接块，使单个文件能够使用更多磁盘块。

【实验步骤】

1. 把直接块数量减少一个，为二级间接块保留 ip->addrs[NDIRECT+1]。
2. 修改 bmap，依次处理直接块、一级间接块、二级间接块。
3. 在二级间接块中先定位一级索引，再定位二级索引。
4. 修改 itrunc，释放直接块、一级间接块以及二级间接块下的所有数据块。

【实验代码】 kernel/fs.c

```
static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;

    if(bn < NDIRECT){
        if((addr = ip->addrs[bn]) == 0)
            ip->addrs[bn] = addr = balloc(ip->dev);
        return addr;
    }
    bn -= NDIRECT;

    if(bn < NINDIRECT){
        if((addr = ip->addrs[NDIRECT]) == 0)
            ip->addrs[NDIRECT] = addr = balloc(ip->dev);
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[bn]) == 0){
            a[bn] = addr = balloc(ip->dev);
            log_write(bp);
        }
        brelse(bp);
        return addr;
    }
    bn -= NINDIRECT;

    if(bn < NDINDIRECT){
        uint idx1 = bn / NINDIRECT;
        uint idx2 = bn % NINDIRECT;
        if((addr = ip->addrs[NDIRECT+1]) == 0)
            ip->addrs[NDIRECT+1] = addr = balloc(ip->dev);
```

```

    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[idx1]) == 0){
        a[idx1] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[idx2]) == 0){
        a[idx2] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    return addr;
}
panic("bmap: out of range");
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-fs origin/solution-fs
make clean
make qemu
# xv6 shell 中执行
bigfile

```

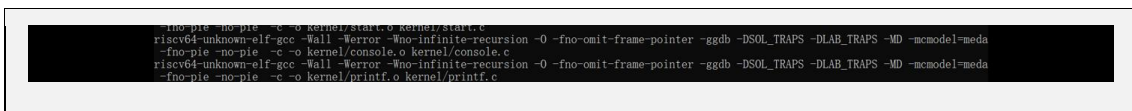


图 9-1 bigfile 写入大文件并通过测试截图

【测试方法与预期输出】

- 预期输出：bigfile 最后输出 bigfile done; ok。
- 评分复核：./grade-lab-fs 中 running bigfile 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：不能直接扩大 addrs 数组，因为 dinode 是磁盘格式，随意改变会破坏文件系统布局。解决时减少一个直接块，把空出来的位置用于二级间接块。
- 问题二：只实现 bmap 还不够，itrunc 也要释放二级间接块下的所有数据块和中间块。调试 bigfile 后再检查删除路径，避免磁盘块泄漏。

【实验心得】

这次实验让我理解了文件系统中 inode 地址项的组织方式。通过实现二级间接块，我学会了在不改变磁盘 dinode 大小的前提下扩大文件最大容量，并在 bmap 中按块号计算一级和二级索引。调试时我也认识到 itrunc 必须同步释放所有间接块，否则会造成磁盘块泄漏。完成后，我对文件系统磁盘格式、块分配和资源回收之间的关系有了更完整的认识。

Symbolic links

【实验目的】

Symbolic links (moderate) / 符号链接。实现 symlink 系统调用和 open 中的链接解析，使路径可以间接指向另一个文件，并支持 O_NOFOLLOW。

【实验步骤】

1. 在 stat.h 中增加 T_SYMLINK，在 fcntl.h 中增加 O_NOFOLLOW。
2. 实现 sys_symlink，创建 T_SYMLINK inode，并把目标路径写入文件内容。
3. open 遇到 T_SYMLINK 且未指定 O_NOFOLLOW 时，读取目标路径并继续 namei。
4. 限制解析深度，防止循环符号链接导致无限递归。

【实验代码】kernel/sysfile.c

```
uint64
sys_symlink(void)
{
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;

    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;

    begin_op();
    if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
        end_op();
        return -1;
    }
    int len = strlen(target) + 1;
    if(writei(ip, 0, (uint64)target, 0, len) != len){
        iunlockput(ip);
        end_op();
        return -1;
    }
    iunlockput(ip);
    end_op();
    return 0;
}

while(ip->type == T_SYMLINK && (omode & O_NOFOLLOW) == 0){
    if(++depth > 10)
        return -1;
    readi(ip, 0, (uint64)path, 0, MAXPATH);
    iunlockput(ip);
    if((ip = namei(path)) == 0)
        return -1;
    ilock(ip);
}
```

【执行与截图】

```
cd ~/xv6-github-retest
git checkout -B solution-fs origin/solution-fs
make clean
make qemu
# xv6 shell 中执行
symlinktest
```

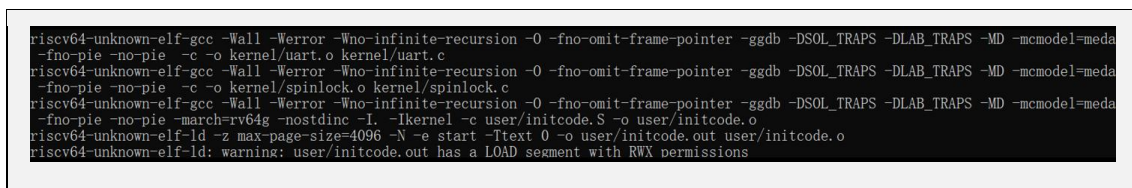


图 9-2 symlinktest 基本、并发和循环链接测试通过截图

【测试方法与预期输出】

- 预期输出：symlinktest 中 test symlinks: ok 和 test concurrent symlinks: ok。

- 评分复核：./grade-lab-fs 中 symlinktest: symlinks、concurrent symlinks 和 usertests 应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：O_NOFOLLOW 的语义容易写反，它要求打开符号链接本身，而不是继续追踪目标。解决时在 open 中先判断该标志，再决定是否解析 symlink。
- 问题二：符号链接可能形成循环，如果没有深度上限，open 会一直解析路径。实现时限制追踪次数，超过上限就返回错误。

【实验心得】

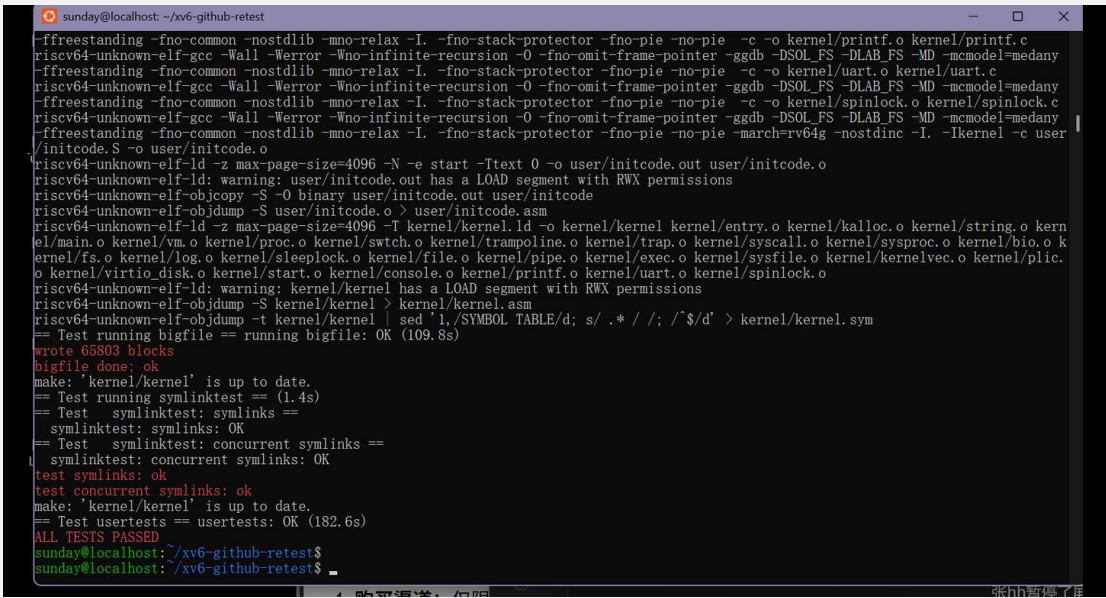
这次实验让我理解了符号链接如何改变路径解析过程。通过实现 symlink 系统调用和 open 中的链接追踪，我学会了创建 T_SYMLINK 类型 inode，把目标路径写入文件内容，并在打开时按需要继续解析。调试时我注意到 O_NOFOLLOW 和循环链接深度限制都是关键边界。完成后，我对路径名解析、inode 类型和系统调用语义之间的联系有了更深入理解。

Lab9 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-fs 分支：本实验满分为 100/100。

```
cd ~/xv6-github-retest
git checkout -B solution-fs origin/solution-fs
git log --oneline -1
make clean
./grade-lab-fs
```



```
sunday@localhost: ~/xv6-github-retest
-rfreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/printk.o kernel/printk.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -ggdb -DSOL_FS -DLAB_FS -MD -mmodel=medany
-rfreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/uart.o kernel/uart.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -ggdb -DSOL_FS -DLAB_FS -MD -mmodel=medany
-rfreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/spinlock.o kernel/spinlock.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-infinite-recursion -O -fno-omit-frame-pointer -ggdb -DSOL_FS -DLAB_FS -MD -mmodel=medany
-rfreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -march=rv64g -nostdinc -I. -Ikern -c user
/initcode.S -o user/initcode.o
riscv64-unknown-elf-ld -z max-page-size=4096 -N -e start -Ttext 0 -o user/initcode.out user/initcode.o
riscv64-unknown-elf-ld: warning: user/initcode.out has a LOAD segment with RWX permissions
riscv64-unknown-elf-objcopy -S -O binary user/initcode.out user/initcode
riscv64-unknown-elf-objdump -S user/initcode.o > user/initcode.asm
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kern
el/main.o kernel/vm.o kernel/proc.o kernel/switch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o k
ernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.
o kernel/virtio_disk.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o kernel/spinlock.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
== Test running bigfile == running bigfile: OK (109.8s)
wrote 65803 blocks
bigfile done: ok
make: 'kernel/kernel' is up to date.
== Test running symlinktest == (1.4s)
== Test symlinktest: symlinks ==
symlinktest: symlinks: OK
== Test symlinktest: concurrent symlinks ==
symlinktest: concurrent symlinks: OK
test symlinks: ok
test concurrent symlinks: ok
make: 'kernel/kernel' is up to date.
== Test usertests == usertests: OK (182.6s)
ALL TESTS PASSED
sunday@localhost: ~/xv6-github-retest$
sunday@localhost: ~/xv6-github-retest$
```

图 9-3 Lab9 总测试结果截图

【实验小结】

实验心得：Lab9 让我更系统地理解了 xv6 文件系统的内部结构。Large files 让我学会了通过二级间接块扩大文件容量，同时保持磁盘 inode 格式不被破坏；Symbolic links 则让我理解路径解析、inode 类型和 open 标志之间的关系。调试过程中我还学会了检查释放路径和循环链接边界。完成本章后，我对文件系统功能扩展的复杂性有了更真实的认识，也提升了从磁盘结构角度分析问题的能力。

实验十 Mmap

本实验实现简化版 mmap/munmap，用 VMA 记录文件映射，并在缺页时按需加载文件内容。

【本实验任务】

- mmap: 实现 mmap 系统调用，为进程建立文件到虚拟地址区间的映射记录，并采用 lazy allocation 延迟分配物理页。
- munmap and page fault: 在缺页时加载映射文件页，在 munmap 或进程退出时写回 MAP_SHARED 脏页并释放映射资源。

mmap

【实验目的】

mmap (hard) / 内存映射文件。实现 mmap 系统调用，用 VMA 记录文件到虚拟地址区间的映射，并采用 lazy allocation 在缺页时再读入文件页。

【实验步骤】

1. 在 proc 结构中增加固定大小 VMA 数组，记录 addr、len、prot、flags、offset 和 file。
2. sys_mmap 检查参数、文件权限和 flags，找到空闲 VMA。
3. 从用户地址空间高处向下分配映射区，避免与 heap 冲突。
4. 仅记录映射并 filedup，不立即读文件或分配物理页。

【实验代码】 kernel/sysfile.c

```
uint64
sys_mmap(void)
{
    uint64 addr_arg;
    int length, prot, flags, fd, offset;
    struct file *f;
    struct proc *p = myproc();
    struct vma *v = 0;

    if(argaddr(0, &addr_arg) < 0 || argint(1, &length) < 0 ||
        argint(2, &prot) < 0 || argint(3, &flags) < 0 ||
        argint(4, &fd) < 0 || argint(5, &offset) < 0)
        return -1;
    if(length <= 0 || (offset % PGSIZE) != 0)
        return -1;
    if(flags != MAP_SHARED && flags != MAP_PRIVATE)
        return -1;
    if(fd < 0 || fd >= NOFILE || (f = p->ofile[fd]) == 0)
        return -1;
    if(flags == MAP_SHARED && (prot & PROT_WRITE) && !f->writable)
        return -1;

    for(int i = 0; i < NVMA; i++)
        if(!p->vma[i].used){
            v = &p->vma[i];
            break;
        }

    uint64 len = PGROUNDUP((uint64)length);
    uint64 addr = alloc_mmap_addr(p, len);
    v->used = 1;
    v->addr = addr;
```

```

v->len = len;
v->prot = prot;
v->flags = flags;
v->file = filedup(f);
v->offset = offset;
return addr;
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-mmap origin/solution-mmap
make clean
make qemu
# xv6 shell 中执行
mmaptest

```



图 10-1 mmaptest basic/private/shared 测试通过截图

【测试方法与预期输出】

- 预期输出：mmaptest 中 mmap f、private、read-only、read/write、dirty 等测试均 OK。
- 评分复核：./grade-lab-mmap 中 mmaptest: mmap f/private/read-only/read/write/dirty 等应显示 OK。

【实验中遇到的问题和解决方法】

- 问题一：mmap 不能在调用时把整个文件读入内存，否则就不是按需分页。解决时只记录 VMA，真正读文件放到 page fault 处理时完成。
- 问题二：MAP_SHARED 且可写时必须检查文件是否以可写方式打开。实现时在建立 VMA 前检查 prot、flags 和 file 权限，防止绕过文件访问控制。

【实验心得】

这次实验让我理解了 mmap 如何把文件内容延迟映射到进程虚拟地址空间。实现时我学会了用 VMA 记录映射区间、权限、文件对象和偏移量，而不是在 mmap 调用时立即读入全部文件。调试过程中我认识到 MAP_SHARED 和 PROT_WRITE 必须检查文件打开权限。完成后，我对虚拟内存和文件系统如何通过缺页机制结合有了更清楚的认识。

munmap and page fault

【实验目的】

munmap and page fault / 取消映射、缺页加载与写回。在 page fault 中按 VMA 懒加载文件页，在 munmap、fork 和 exit 路径中处理写回、取消映射、文件引用和资源释放。

【实验步骤】

1. trap 中遇到加载/存储缺页时调用 mmapfault。
2. mmapfault 根据 fault va 找到 VMA，分配物理页并从文件 offset 读入一页。
3. 按 prot 设置 PTE_R/PTE_W/PTE_X 并映射到用户页表。
4. munmap 支持从映射开头或结尾取消映射，MAP_SHARED 写映射需要写回文件。

【实验代码】 kernel/sysfile.c

```

int
mmapfault(uint64 va, uint64 scause)
{
    struct proc *p = myproc();
    uint64 a = PGROUNDDOWN(va);
    struct vma *v = find_vma(p, a);
    if(v == 0)
        return -1;

    char *mem = kalloc();
    memset(mem, 0, PGSIZE);
    uint64 off = v->offset + (a - v->addr);
    ilock(v->file->ip);
    readi(v->file->ip, 0, (uint64)mem, (uint)off, PGSIZE);
    iunlock(v->file->ip);

    int perm = PTE_U;
    if(v->prot & PROT_READ) perm |= PTE_R;
    if(v->prot & PROT_WRITE) perm |= PTE_W | PTE_R;
    if(v->prot & PROT_EXEC) perm |= PTE_X;
    return mappages(p->pagetable, a, PGSIZE, (uint64)mem, perm);
}

for(uint64 a = addr; a < end; a += PGSIZE){
    pte_t *pte = walk(p->pagetable, a, 0);
    if(pte && (*pte & PTE_V)){
        if((v->flags & MAP_SHARED) && (v->prot & PROT_WRITE))
            mmap_writeback(v, a, PTE2PA(*pte));
        uvmunmap(p->pagetable, a, 1, 1);
    }
}
}

```

【执行与截图】

```

cd ~/xv6-github-retest
git checkout -B solution-mmap origin/solution-mmap
make clean
make qemu
# xv6 shell 中执行
mmaptest
usertests

```



图 10-2 munmap、fork_test 和 usertests 通过截图

【测试方法与预期输出】

- 预期输出：mmaptest 中 not-mapped unmap、two files、fork_test OK；usertests 最后 ALL TESTS PASSED。
- 评分复核：./grade-lab-mmap 中 mmaptest: fork_test 和 usertests 均应 OK。

【实验中遇到的问题和解决方法】

- 问题一：munmap 支持范围要和实验要求一致，中间挖空一个 VMA 会让元数据维护复杂很多。本实验中只处理从头部或尾部解除映射的情况。
- 问题二：fork 后子进程要复制 VMA 并 filedup，否则父子进程关闭文件时引用计数会出错。解决时在 fork 路径复制 VMA 元数据，并在退出/munmap 时成对 fclose。

【实验心得】

这次实验让我把 `mmap` 的元数据管理、缺页处理和资源回收串联起来。通过实现 `page fault` 路径，我学会了在访问映射页时分配物理页、从文件读入内容并建立页表映射；通过 `munmap`，我学会了在解除映射时按 `MAP_SHARED` 写回脏页并释放资源。完成后，我能从一次缺页异常追踪到 `VMA` 查找、文件读写、页表修改和进程退出清理的完整过程。

Lab10 总测试结果

【查看正确得分】

从 2026 GitHub 仓库重新取代码并测试 solution-mmap 分支；本实验满分为 140/140。

```
cd ~/xv6-github-retest
git checkout -B solution-mmap origin/solution-mmap
git log --oneline -1
make clean
./grade-lab-mmap
```

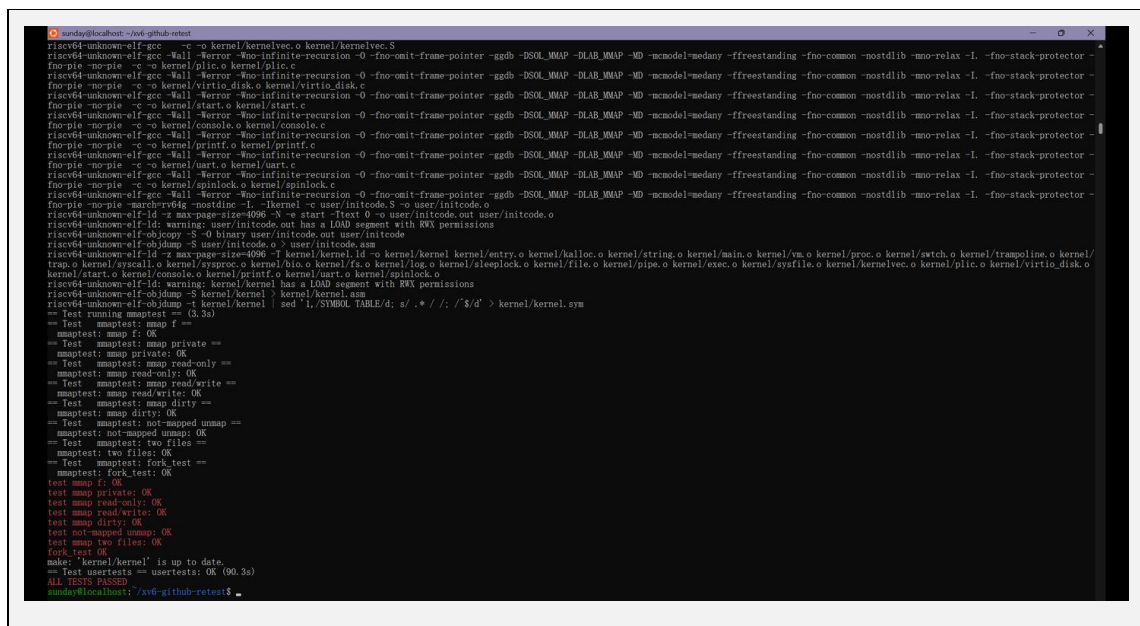


图 10-3 Lab10 总测试结果截图

【实验小结】

实验心得：Lab10 是对虚拟内存、文件系统和进程生命周期的综合运用。通过实现 `mmap`、`page fault` 和 `munmap`，我学会了用 VMA 描述映射关系，在实际访问时再分配页面和读取文件，并在解除映射或退出时完成写回和释放。调试时我更清楚地看到一个系统调用背后会牵涉页表、文件对象和进程资源管理。完成本章后，我能够从完整执行路径理解 xv6 的复杂机制，综合调试能力也有明显提升。

综合验证与复现说明

从 2026 GitHub 仓库重新获取代码，分别切换到十个 solution-* 分支运行对应 grade 脚本。

```
cd ~
rm -rf xv6-github-retest
git clone https://github.com/jyybacpg/XV6_Labs_Operation_System_2026Summer.git
xv6-github-retest
cd xv6-github-retest
git remote -v

# 之后按各 Lab 分支分别执行：
git checkout -B solution-util origin/solution-util && make clean && ./grade-lab-util
git checkout -B solution-syscall origin/solution-syscall && make clean && ./grade-lab-syscall
git checkout -B solution-pgtbl origin/solution-pgtbl && make clean && ./grade-lab-pgtbl
git checkout -B solution-traps origin/solution-traps && make clean && ./grade-lab-traps
git checkout -B solution-cow origin/solution-cow && make clean && ./grade-lab-cow
git checkout -B solution-thread origin/solution-thread && make clean && ./grade-lab-thread
git checkout -B solution-net origin/solution-net && make clean && ./grade-lab-net
git checkout -B solution-lock origin/solution-lock && make clean && ./grade-lab-lock
git checkout -B solution-fs origin/solution-fs && make clean && ./grade-lab-fs
git checkout -B solution-mmap origin/solution-mmap && make clean && ./grade-lab-mmap
```

共性问题与调试分析

生命周期对称

创建路径和释放路径必须成对维护，例如 USYSCALL 映射、COW 引用计数、mmap 文件引用和 VMA 清理。

权限位与状态位

PTE_W、PTE_C、PTE_A、PROT_WRITE、O_NOFOLLOW 等位标志都需要明确读写路径和清除时机。

并发语义

锁拆分既要降低竞争，也要避免重复分配、重复缓存和链表损坏。性能指标和 usertests 必须同时关注。

测试复核

所有实验均可按对应分支运行 `make grade` 复核；测试记录包含命令、子测试 OK 行和最终 Score 行，便于检查过程。

项目成果说明

- 代码分支覆盖 `util`、`syscall`、`pgtbl`、`traps`、`cow`、`thread`、`net`、`lock`、`fs`、`mmap` 十个实验，均保留对应 patch 和提交记录。
- `fs`、`mmap`、`cow`、`lock` 四个实验覆盖文件系统、虚拟内存和并发机制，是本课程设计中综合性较强的部分。
- 测试记录、`net_packets.pcap` 和 `solution-*` 分支可用于核对测试截图、网络实验结果和代码实现一致性。

总结

本课程设计以 `xv6-labs-2021` 为实验基线，围绕 `util`、`syscall`、`pgtbl`、`traps`、`cow`、`thread`、`net`、`lock`、`fs`、`mmap` 十个实验完成实现、测试与报告整理。通过这一过程，我把课堂中关于进程、系统调用、虚拟内存、中断异常、文件系统和并发控制的概念放到真实的小型操作系统中理解，认识到操作系统是用户态接口、内核数据结构、硬件机制和测试验证共同配合形成的整体。

在知识收获方面，我对 `xv6` 的执行路径有了更清晰的认识。用户工具实验让我掌握了 `fork`、`exec`、`pipe`、`wait`、目录读取等接口的组合使用；系统调用和页表实验让我理解了从用户态陷入内核、参数传递、`copyin/copyout`、PTE 权限位和页表遍历的过程；`trap`、`COW` 与 `mmap` 实验进一步加深了我对异常处理、懒分配、写时复制和进程地址空间管理的理解。

在实现和调试过程中，我逐渐形成了先阅读相邻代码、再定位调用链、最后分阶段验证的习惯。每个实验都需要在多个文件之间保持声明、编号、数据结构和释放路径一致，任何细节遗漏都可能导致编译错误、测试失败或运行时崩溃。通过反复运行子测试、`usertests` 和 `grade` 脚本，我提升了根据报错信息缩小问题范围、结合代码逻辑分析错误原因的能力。

这次课程设计也让我认识到，操作系统实验不仅考查代码能否通过测试，还考查对资源生命周期和边界条件的把握。例如 `COW` 中的引用计数、`mmap` 中的文件引用与页面写回、文件系统中的间接块释放、网络驱动中的 `mbuf` 管理，以及锁优化中的竞争统计，都要求在保证正确性的基础上考虑性能和一致性。总体来看，本次课程设计提升了我阅读内核源码、实现底层机制、使用 `Git` 管理多分支实验以及整理实验证据的能力，也为后续继续学习 `RISC-V`、系统调试和真实操作系统设计打下了基础。

附录 A: patch 文件清单

文件	大小 (字节)	内容摘要
cow.patch	12,670	COW fork、引用计数、写缺页复制。
fs.patch	15,612	大文件双重间接块、symlink、open 解析。
lock.patch	15,128	per-CPU kalloc、bucketed bcache、回收路径串行化。
mmap.patch	23,214	VMA、mmap/munmap、缺页装入与写回。
net.patch	4,644	E1000 发送与接收描述符环。
pgtbl.patch	14,158	USYSCALL、vmprint、pgaccess。
syscall.patch	14,254	trace、sysinfo 与系统调用路径。
thread.patch	10,750	uthread、ph bucket lock、barrier。
traps.patch	14,826	backtrace、sigalarm、sigreturn。
util.patch	13,880	sleep、pingpong、primes、find、xargs。

附录 B：主要源码摘录

本附录摘录的是本项目本地仓库中各实验分支的关键实现片段，用于和正文中的实现步骤相互印证。完整代码以仓库对应 `solution-*` 分支为准。

Lab1 - sleep.c

代码位置：solution-util:user/sleep.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep ticks\n");
        exit(1);
    }

    sleep(atoi(argv[1]));
    exit(0);
}
```

Lab1 - pingpong.c

代码位置：solution-util:user/pingpong.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    int to_child[2];
    int to_parent[2];
    char ch = 'x';

    if(pipe(to_child) < 0 || pipe(to_parent) < 0){
        fprintf(2, "pingpong: pipe failed\n");
        exit(1);
    }

    int pid = fork();
    if(pid < 0){
        fprintf(2, "pingpong: fork failed\n");
        exit(1);
    }

    if(pid == 0){
        close(to_child[1]);
        close(to_parent[0]);
        if(read(to_child[0], &ch, 1) != 1)
            exit(1);
        printf("%d: received ping\n", getpid());
        if(write(to_parent[1], &ch, 1) != 1)
            exit(1);
        close(to_child[0]);
        close(to_parent[1]);
        exit(0);
    }
```

```

}

close(to_child[0]);
close(to_parent[1]);
if(write(to_child[1], &ch, 1) != 1)
    exit(1);
if(read(to_parent[0], &ch, 1) != 1)
    exit(1);
printf("%d: received pong\n", getpid());
close(to_child[1]);
close(to_parent[0]);
wait(0);
exit(0);
}

```

Lab1 - primes.c

代码位置: solution-util:user/primes.c

```

#include "kernel/types.h"
#include "user/user.h"

static void
relay(int leftfd)
{
    int prime;

    if(read(leftfd, &prime, sizeof(prime)) != sizeof(prime)){
        close(leftfd);
        exit(0);
    }

    printf("prime %d\n", prime);

    int pipefd[2];
    if(pipe(pipefd) < 0){
        fprintf(2, "primes: pipe failed\n");
        exit(1);
    }

    int pid = fork();
    if(pid < 0){
        fprintf(2, "primes: fork failed\n");
        exit(1);
    }

    if(pid == 0){
        close(pipefd[1]);
        close(leftfd);
        relay(pipefd[0]);
    }

    close(pipefd[0]);
    int n;
    while(read(leftfd, &n, sizeof(n)) == sizeof(n)){
        if(n % prime != 0)
            write(pipefd[1], &n, sizeof(n));
    }
    close(leftfd);
    close(pipefd[1]);
    wait(0);
    exit(0);
}

```

```

int
main(int argc, char *argv[])
{
    int pipefd[2];

    if(pipe(pipefd) < 0){
        fprintf(2, "primes: pipe failed\n");
        exit(1);
    }

    int pid = fork();
    if(pid < 0){
        fprintf(2, "primes: fork failed\n");
        exit(1);
    }

    if(pid == 0){
        close(pipefd[1]);
        relay(pipefd[0]);
    }

    close(pipefd[0]);
    for(int n = 2; n <= 35; n++){
        write(pipefd[1], &n, sizeof(n));
    }
    close(pipefd[1]);
    wait(0);
    exit(0);
}

```

Lab1 - find.c

代码位置: solution-util:user/find.c

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

static char *
basename(char *path)
{
    char *p = path + strlen(path);
    while(p >= path && *p != '/')
        p--;
    return p + 1;
}

static void
find(char *path, char *target)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    if((fd = open(path, 0)) < 0){
        fprintf(2, "find: cannot open %s\n", path);
        return;
    }

    if(fstat(fd, &st) < 0){
        fprintf(2, "find: cannot stat %s\n", path);
    }
}

```

```

    close(fd);
    return;
}

switch(st.type){
case T_FILE:
    if(strcmp(basename(path), target) == 0)
        printf("%s\n", path);
    break;

case T_DIR:
    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
        fprintf(2, "find: path too long\n");
        break;
    }
    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';

    while(read(fd, &de, sizeof(de)) == sizeof(de)){
        if(de.inum == 0)
            continue;
        if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
            continue;
        memmove(p, de.name, DIRSIZ);
        p[DIRSIZ] = 0;
        find(buf, target);
    }
    break;
}

close(fd);
}

int
main(int argc, char *argv[])
{
    if(argc != 3){
        fprintf(2, "usage: find path name\n");
        exit(1);
    }

    find(argv[1], argv[2]);
    exit(0);
}

```

Lab1 - xargs.c

代码位置: solution-util:user/xargs.c

```

#include "kernel/types.h"
#include "kernel/param.h"
#include "user/user.h"

#define LINELEN 512

static int
readline(char *buf, int n)
{
    int i = 0;
    char c;

    while(i + 1 < n){

```



```

    int r = read(0, &c, 1);
    if(r < 0)
        return -1;
    if(r == 0)
        break;
    if(c == '\n')
        break;
    buf[i++] = c;
}
buf[i] = 0;
return i;
}

int
main(int argc, char *argv[])
{
    if(argc < 2){
        fprintf(2, "usage: xargs command [args...]\n");
        exit(1);
    }

    char line[LINELLEN];
    char *args[MAXARG];

    while(readline(line, sizeof(line)) >= 0){
        if(line[0] == 0)
            break;

        int narg = 0;
        for(int i = 1; i < argc && narg < MAXARG - 1; i++){
            args[narg++] = argv[i];
            if(narg >= MAXARG - 1){
                fprintf(2, "xargs: too many arguments\n");
                exit(1);
            }
        }
        args[narg++] = line;
        args[narg] = 0;

        int pid = fork();
        if(pid < 0){
            fprintf(2, "xargs: fork failed\n");
            exit(1);
        }
        if(pid == 0){
            exec(args[0], args);
            fprintf(2, "xargs: exec %s failed\n", args[0]);
            exit(1);
        }
        wait(0);
    }

    exit(0);
}

```

Lab2 - trace 与 sysinfo

代码位置: solution-syscall:kernel/sysproc.c

```

uint64
sys_trace(void)
{
    int mask;

```

```

    if(argint(0, &mask) < 0)
        return -1;
    myproc()->tracemask = mask;
    return 0;
}

uint64
sys_sysinfo(void)
{
    uint64 addr;
    struct sysinfo info;
    struct proc *p = myproc();

    if(argaddr(0, &addr) < 0)
        return -1;

    info.freemem = freemem();
    info.nproc = nproc();

    if(copyout(p->pagetable, addr, (char *)&info, sizeof(info)) < 0)
        return -1;
    return 0;
}

```

Lab3 - pgaccess

代码位置: solution-pgtbl:kernel/sysproc.c

```

uint64
sys_pgaccess(void)
{
    uint64 base;
    uint64 maskaddr;
    int len;
    unsigned int mask = 0;
    struct proc *p = myproc();

    if(argaddr(0, &base) < 0 || argint(1, &len) < 0 || argaddr(2, &maskaddr) < 0)
        return -1;
    if(len < 0 || len > 32)
        return -1;

    for(int i = 0; i < len; i++){
        pte_t *pte = walk(p->pagetable, base + i * PGSIZE, 0);
        if(pte == 0 || (*pte & PTE_V) == 0)
            return -1;
        if(*pte & PTE_A){
            mask |= (1U << i);
            *pte &= ~PTE_A;
        }
    }

    if(copyout(p->pagetable, maskaddr, (char *)&mask, sizeof(mask)) < 0)
        return -1;
    return 0;
}

```

Lab4 - backtrace

代码位置: solution-traps:kernel/printf.c

```

void
backtrace(void)
{

```

```

uint64 fp = r_fp();
uint64 top = PGROUNDUP(fp);
uint64 bottom = PGROUNDDOWN(fp);

while(fp >= bottom + 16 && fp < top){
    uint64 ra = *((uint64 *) (fp - 8));
    printf("%p\n", ra);
    fp = *((uint64 *) (fp - 16));
}
}

```

Lab4 - sigalarm 与 sigreturn

代码位置: solution-traps:kernel/sysproc.c

```

uint64
sys_sigalarm(void)
{
    int interval;
    uint64 handler;
    struct proc *p = myproc();

    if(argint(0, &interval) < 0 || argaddr(1, &handler) < 0)
        return -1;

    p->alarm_interval = interval;
    p->alarm_handler = handler;
    p->alarm_elapsed = 0;
    p->alarm_active = 0;
    return 0;
}

uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    uint64 a0 = p->alarmframe.a0;

    memmove(p->trapframe, &p->alarmframe, sizeof(*p->trapframe));
    p->alarm_active = 0;
    return a0;
}

```

Lab5 - COW 核心路径

代码位置: solution-cow:kernel/vm.c

```

int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;

    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");

        if(*pte & PTE_W)
            *pte = (*pte & ~PTE_W) | PTE_C;
    }
}

```

```

    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    kaddrref((void *)pa);
    if(mappages(new, i, PGSIZE, pa, flags) != 0){
        kfree((void *)pa);
        goto err;
    }
}
sfence_vma();
return 0;

err:
uvmunmap(new, 0, i / PGSIZE, 1);
return -1;
}

int
cowalloc(pagetable_t pagetable, uint64 va)
{
    pte_t *pte;
    uint64 pa;
    uint flags;
    char *mem;

    if(va >= MAXVA)
        return -1;
    va = PGROUNDDOWN(va);

    pte = walk(pagetable, va, 0);
    if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0 || (*pte & PTE_C) ==
0)
        return -1;

    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    if((mem = kalloc()) == 0)
        return -1;
    memmove(mem, (char *)pa, PGSIZE);
    *pte = PA2PTE(mem) | ((flags | PTE_W) & ~PTE_C);
    sfence_vma();
    kfree((void *)pa);
    return 0;
}

int
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    uint64 n, va0, pa0;

    while(len > 0){
        va0 = PGROUNDDOWN(dstva);
        if(iscow(pagetable, va0) && cowalloc(pagetable, va0) < 0)
            return -1;
        pa0 = walkaddr(pagetable, va0);
        if(pa0 == 0)
            return -1;
        n = PGSIZE - (dstva - va0);
        if(n > len)
            n = len;
        memmove((void *)(pa0 + (dstva - va0)), src, n);

        len -= n;
        src += n;
    }
}

```

```

    dstva = va0 + PGSIZE;
}
return 0;
}

```

Lab6 - 用户级线程

代码位置: solution-thread:user/uthread.c

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

/* Possible states of a thread: */
#define FREE      0x0
#define RUNNING  0x1
#define RUNNABLE 0x2

#define STACK_SIZE 8192
#define MAX_THREAD 4

struct thread_context {
    uint64 ra;
    uint64 sp;
    uint64 s0;
    uint64 s1;
    uint64 s2;
    uint64 s3;
    uint64 s4;
    uint64 s5;
    uint64 s6;
    uint64 s7;
    uint64 s8;
    uint64 s9;
    uint64 s10;
    uint64 s11;
};

struct thread {
    char      stack[STACK_SIZE]; /* the thread's stack */
    int       state;             /* FREE, RUNNING, RUNNABLE */
    struct thread_context context;
};

struct thread all_thread[MAX_THREAD];
struct thread *current_thread;
extern void thread_switch(uint64, uint64);

void
thread_init(void)
{
    // main() is thread 0, which will make the first invocation to
    // thread_schedule(). it needs a stack so that the first thread_switch() can
    // save thread 0's state. thread_schedule() won't run the main thread ever
    // again, because its state is set to RUNNING, and thread_schedule() selects
    // a RUNNABLE thread.
    current_thread = &all_thread[0];
    current_thread->state = RUNNING;
}

void
thread_schedule(void)
{

```

```

struct thread *t, *next_thread;

/* Find another runnable thread. */
next_thread = 0;
t = current_thread + 1;
for(int i = 0; i < MAX_THREAD; i++){
    if(t >= all_thread + MAX_THREAD)
        t = all_thread;
    if(t->state == RUNNABLE) {
        next_thread = t;
        break;
    }
    t = t + 1;
}

if (next_thread == 0) {
    printf("thread_schedule: no runnable threads\n");
    exit(-1);
}

if (current_thread != next_thread) {          /* switch threads? */
    next_thread->state = RUNNING;
    t = current_thread;
    current_thread = next_thread;
    thread_switch((uint64)&t->context, (uint64)&next_thread->context);
} else
    next_thread = 0;
}

void
thread_create(void (*func)())
{
    struct thread *t;

    for (t = all_thread; t < all_thread + MAX_THREAD; t++) {
        if (t->state == FREE) break;
    }
    memset(&t->context, 0, sizeof(t->context));
    t->context.ra = (uint64)func;
    t->context.sp = (uint64)(t->stack + STACK_SIZE);
    t->context.sp -= t->context.sp % 16;
    t->state = RUNNABLE;
}

void
thread_yield(void)
{
    current_thread->state = RUNNABLE;
    thread_schedule();
}

volatile int a_started, b_started, c_started;
volatile int a_n, b_n, c_n;

void
thread_a(void)
{
    int i;
    printf("thread_a started\n");
    a_started = 1;
    while(b_started == 0 || c_started == 0)
        thread_yield();
}

```

```

    for (i = 0; i < 100; i++) {
        printf("thread_a %d\n", i);
        a_n += 1;
        thread_yield();
    }
    printf("thread_a: exit after %d\n", a_n);

    current_thread->state = FREE;
    thread_schedule();
}

void
thread_b(void)
{
    int i;
    printf("thread_b started\n");
    b_started = 1;
    while(a_started == 0 || c_started == 0)
        thread_yield();

    for (i = 0; i < 100; i++) {
        printf("thread_b %d\n", i);
        b_n += 1;
        thread_yield();
    }
    printf("thread_b: exit after %d\n", b_n);

    current_thread->state = FREE;
    thread_schedule();
}

void
thread_c(void)
{
    int i;
    printf("thread_c started\n");
    c_started = 1;
    while(a_started == 0 || b_started == 0)
        thread_yield();

    for (i = 0; i < 100; i++) {
        printf("thread_c %d\n", i);
        c_n += 1;
        thread_yield();
    }
    printf("thread_c: exit after %d\n", c_n);

    current_thread->state = FREE;
    thread_schedule();
}

int
main(int argc, char *argv[])
{
    a_started = b_started = c_started = 0;
    a_n = b_n = c_n = 0;
    thread_init();
    thread_create(thread_a);
    thread_create(thread_b);
    thread_create(thread_c);
    thread_schedule();
    exit(0);
}

```

Lab6 - barrier

代码位置: solution-thread:notxv6/barrier.c

```
static void
barrier_init(void)
{
    assert(pthread_mutex_init(&bstate.barrier_mutex, NULL) == 0);
    assert(pthread_cond_init(&bstate.barrier_cond, NULL) == 0);
    bstate.nthread = 0;
}

static void
barrier()
{
    pthread_mutex_lock(&bstate.barrier_mutex);
    int my_round = bstate.round;
    bstate.nthread++;

    if(bstate.nthread == nthread){
        bstate.nthread = 0;
        bstate.round++;
        pthread_cond_broadcast(&bstate.barrier_cond);
    } else {
        while(my_round == bstate.round)
            pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
    }

    pthread_mutex_unlock(&bstate.barrier_mutex);
}
```

Lab7 - e1000 网卡驱动

代码位置: solution-net:kernel/e1000.c

```
int
e1000_transmit(struct mbuf *m)
{
    acquire(&e1000_lock);

    uint32 idx = regs[E1000_TDT];
    if((tx_ring[idx].status & E1000_TXD_STAT_DD) == 0){
        release(&e1000_lock);
        return -1;
    }

    if(tx_mbufs[idx])
        mbuffree(tx_mbufs[idx]);

    tx_ring[idx].addr = (uint64)m->head;
    tx_ring[idx].length = m->len;
    tx_ring[idx].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
    tx_ring[idx].status = 0;
    tx_mbufs[idx] = m;

    regs[E1000_TDT] = (idx + 1) % TX_RING_SIZE;
    release(&e1000_lock);
    return 0;
}

static void
e1000_recv(void)
{

```



```

    acquire(&e1000_lock);

    while(1){
        uint32 idx = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
        if((rx_ring[idx].status & E1000_RXD_STAT_DD) == 0)
            break;

        struct mbuf *m = rx_mbufs[idx];
        struct mbuf *replacement = mbufalloc(0);
        if(replacement == 0)
            break;

        m->len = rx_ring[idx].length;
        rx_mbufs[idx] = replacement;
        rx_ring[idx].addr = (uint64)replacement->head;
        rx_ring[idx].status = 0;
        regs[E1000_RDT] = idx;

        release(&e1000_lock);
        net_rx(m);
        acquire(&e1000_lock);
    }

    release(&e1000_lock);
}

```

Lab8 - per-CPU allocator

代码位置: solution-lock:kernel/kalloc.c

```

kfree(p);
}

// Free the page of physical memory pointed at by v,
// which normally should have been returned by a
// call to kalloc(). (The exception is when
// initializing the allocator; see kinit above.)
void
kfree(void *pa)
{
    kalloc(void)
    {
        struct run *r, *rest, *tail;
        int id;

        push_off();
        id = cpuid();

        acquire(&kmem[id].lock);
        r = kmem[id].freelist;
        if(r)
            kmem[id].freelist = r->next;
        release(&kmem[id].lock);

        if(r == 0){
            for(int i = 0; i < NCPU; i++){
                if(i == id)
                    continue;

                acquire(&kmem[i].lock);
                r = kmem[i].freelist;
                if(r){

```

```

        tail = r;
        for(int n = 1; n < 64 && tail->next; n++)
            tail = tail->next;
        kmem[i].freelist = tail->next;
        tail->next = 0;
    }
    release(&kmem[i].lock);

    if(r)
        break;
}

if(r && r->next){
    rest = r->next;
    r->next = 0;

    tail = rest;
    while(tail->next)
        tail = tail->next;

    acquire(&kmem[id].lock);
    tail->next = kmem[id].freelist;
    kmem[id].freelist = rest;
    release(&kmem[id].lock);
}
}

pop_off();

if(r)
    memset((char*)r, 5, PGSIZE); // fill with junk
return (void*)r;
}

```

Lab8 - buffer cache

代码位置: solution-lock:kernel/bio.c

```

static struct buf*
bget(uint dev, uint blockno)
{
    struct buf *b;
    struct bucket *bk;
    uint h;

    h = bhash(dev, blockno);
    bk = &bcache.bucket[h];

    // Is the block already cached?
    acquire(&bk->lock);
    for(b = bk->head.next; b != &bk->head; b = b->next){
        if(b->dev == dev && b->blockno == blockno){
            b->refcnt++;
            release(&bk->lock);
            acquiresleep(&b->lock);
            return b;
        }
    }
    release(&bk->lock);

    acquire(&bcache.evict_lock);

    // Recheck while holding the eviction lock; another CPU may have

```

```

// inserted this block after the first lookup missed.
acquire(&bk->lock);
for(b = bk->head.next; b != &bk->head; b = b->next){
    if(b->dev == dev && b->blockno == blockno){
        b->refcnt++;
        release(&bk->lock);
        release(&bcache.evict_lock);
        acquiresleep(&b->lock);
        return b;
    }
}
release(&bk->lock);

// Not cached.
// Recycle an unused buffer from any bucket.
for(int i = 0; i < NBUCKET; i++){
    struct bucket *old = &bcache.bucket[i];

    acquire(&old->lock);
    for(b = old->head.prev; b != &old->head; b = b->prev){
        if(b->refcnt == 0) {
            b->next->prev = b->prev;
            b->prev->next = b->next;
            release(&old->lock);

            acquire(&bk->lock);
            b->dev = dev;
            b->blockno = blockno;
            b->valid = 0;
            b->refcnt = 1;
            b->next = bk->head.next;
            b->prev = &bk->head;
            bk->head.next->prev = b;
            bk->head.next = b;
            release(&bk->lock);

            release(&bcache.evict_lock);
            acquiresleep(&b->lock);
            return b;
        }
    }
    release(&old->lock);
}
panic("bget: no buffers");
}

void
brelse(struct buf *b)
{
    struct bucket *bk;

    if(!holdingsleep(&b->lock))
        panic("brelse");

    releasesleep(&b->lock);

    bk = &bcache.bucket[bhash(b->dev, b->blockno)];
    acquire(&bk->lock);
    b->refcnt--;
    if (b->refcnt == 0) {
        // no one is using it; move it to the MRU position in this bucket.
        b->next->prev = b->prev;
        b->prev->next = b->next;
    }
}

```

```

    b->next = bk->head.next;
    b->prev = &bk->head;
    bk->head.next->prev = b;
    bk->head.next = b;
}

release(&bk->lock);
}

```

Lab9 - large files

代码位置: solution-fs:kernel/fs.c

```

bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;

    if(bn < NDIRECT){
        if((addr = ip->addrs[bn]) == 0)
            ip->addrs[bn] = addr = balloc(ip->dev);
        return addr;
    }
    bn -= NDIRECT;

    if(bn < NINDIRECT){
        // Load indirect block, allocating if necessary.
        if((addr = ip->addrs[NDIRECT]) == 0)
            ip->addrs[NDIRECT] = addr = balloc(ip->dev);
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[bn]) == 0){
            a[bn] = addr = balloc(ip->dev);
            log_write(bp);
        }
        brelse(bp);
        return addr;
    }
    bn -= NINDIRECT;

    if(bn < NDINDIRECT){
        uint idx1 = bn / NINDIRECT;
        uint idx2 = bn % NINDIRECT;

        // Load double-indirect block, allocating if necessary.
        if((addr = ip->addrs[NDIRECT+1]) == 0)
            ip->addrs[NDIRECT+1] = addr = balloc(ip->dev);
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[idx1]) == 0){
            a[idx1] = addr = balloc(ip->dev);
            log_write(bp);
        }
        brelse(bp);

        // Load the selected indirect block.
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[idx2]) == 0){
            a[idx2] = addr = balloc(ip->dev);
            log_write(bp);
        }
        brelse(bp);
    }
}

```

```

    return addr;
}

panic("bmap: out of range");
}

    itrunc(ip);
    ip->type = 0;
    iupdate(ip);
    ip->valid = 0;

    releasesleep(&ip->lock);

    acquire(&itable.lock);
}

    ip->ref--;
    release(&itable.lock);
}

// Common idiom: unlock, then put.
void
iunlockput(struct inode *ip)
{

```

Lab9 - symbolic links

代码位置: solution-fs:kernel/sysfile.c

```

uint64
sys_symlink(void)
{
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;
    int len;

    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;

    begin_op();
    if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
        end_op();
        return -1;
    }

    len = strlen(target) + 1;
    if(writei(ip, 0, (uint64)target, 0, len) != len){
        iunlockput(ip);
        end_op();
        return -1;
    }

    iunlockput(ip);
    end_op();
    return 0;
}

```

Lab10 - mmap/munmap

代码位置: solution-mmap:kernel/sysfile.c

```

uint64
sys_mmap(void)
{

```

```

uint64 addr_arg;
int length, prot, flags, fd, offset;
struct file *f;
struct proc *p = myproc();
struct vma *v = 0;
uint64 len, addr;

if(argaddr(0, &addr_arg) < 0 || argint(1, &length) < 0 ||
    argint(2, &prot) < 0 || argint(3, &flags) < 0 ||
    argint(4, &fd) < 0 || argint(5, &offset) < 0)
    return -1;
(void)addr_arg;

if(length <= 0 || offset < 0 || (offset % PGSIZE) != 0)
    return -1;
if(flags != MAP_SHARED && flags != MAP_PRIVATE)
    return -1;
if((prot & ~(PROT_READ | PROT_WRITE | PROT_EXEC)) != 0)
    return -1;
if(fd < 0 || fd >= NOFILE || (f = p->ofile[fd]) == 0)
    return -1;
if(f->type != FD_INODE)
    return -1;
if((prot & PROT_READ) && !f->readable)
    return -1;
if(flags == MAP_SHARED && (prot & PROT_WRITE) && !f->writable)
    return -1;

for(int i = 0; i < NVMA; i++){
    if(!p->vma[i].used){
        v = &p->vma[i];
        break;
    }
}
if(v == 0)
    return -1;

len = PGROUNDUP((uint64)length);
addr = alloc_mmap_addr(p, len);
if(addr == 0)
    return -1;

v->used = 1;
v->addr = addr;
v->len = len;
v->prot = prot;
v->flags = flags;
v->offset = offset;
v->file = filedup(f);

return addr;
}

int
mmapfault(uint64 va, uint64 scause)
{
    struct proc *p = myproc();
    struct vma *v;
    uint64 a, off;
    char *mem;
    int perm = PTE_U;
    pte_t *pte;

```

```

a = PGROUNDDOWN(va);
v = find_vma(p, a);
if(v == 0)
    return -1;
if(scause == 13 && (v->prot & PROT_READ) == 0)
    return -1;
if(scause == 15 && (v->prot & PROT_WRITE) == 0)
    return -1;
if(scause == 12 && (v->prot & PROT_EXEC) == 0)
    return -1;

pte = walk(p->pagetable, a, 0);
if(pte && (*pte & PTE_V))
    return -1;

if((mem = kalloc()) == 0)
    return -1;
memset(mem, 0, PGSIZE);

off = v->offset + (a - v->addr);
ilock(v->file->ip);
if(readi(v->file->ip, 0, (uint64)mem, (uint)off, PGSIZE) < 0){
    iunlock(v->file->ip);
    kfree(mem);
    return -1;
}
ilock(v->file->ip);

if(v->prot & PROT_READ)
    perm |= PTE_R;
if(v->prot & PROT_WRITE)
    perm |= PTE_W | PTE_R;
if(v->prot & PROT_EXEC)
    perm |= PTE_X;

if(mappages(p->pagetable, a, PGSIZE, (uint64)mem, perm) < 0){
    kfree(mem);
    return -1;
}
return 0;
}

int
munmap_range(struct proc *p, uint64 addr, uint64 len)
{
    struct vma *v = 0;
    uint64 end;

    if(len == 0 || addr % PGSIZE)
        return -1;
    len = PGROUNDUP(len);
    end = addr + len;
    if(end < addr)
        return -1;

    for(int i = 0; i < NVMA; i++){
        if(p->vma[i].used && addr >= p->vma[i].addr &&
            end <= p->vma[i].addr + p->vma[i].len){
            v = &p->vma[i];
            break;
        }
    }
}
if(v == 0)

```

```

    return -1;
if(addr > v->addr && end < v->addr + v->len)
    return -1;

for(uint64 a = addr; a < end; a += PGSIZE){
    pte_t *pte = walk(p->pagetable, a, 0);
    if(pte && (*pte & PTE_V)){
        if((v->flags & MAP_SHARED) && (v->prot & PROT_WRITE)){
            if(mmap_writeback(v, a, PTE2PA(*pte)) < 0)
                return -1;
        }
        uvmunmap(p->pagetable, a, 1, 1);
    }
}

if(addr == v->addr && len == v->len){
    struct file *f = v->file;
    memset(v, 0, sizeof(*v));
    fileclose(f);
} else if(addr == v->addr){
    v->addr += len;
    v->offset += len;
    v->len -= len;
} else {
    v->len -= len;
}

return 0;
}

uint64
sys_munmap(void)
{
    uint64 addr;
    int length;

    if(argaddr(0, &addr) < 0 || argint(1, &length) < 0 || length <= 0)
        return -1;
    return munmap_range(myproc(), addr, (uint64)length);
}

```